

Web-Based AI-Secured User Authentication System: SAHAYAA

A Project Report submitted in partial fulfillment of the requirements for the Award of Degree of

Bachelor of Technology

In

COMPUTER SCIENCE AND ENGINEERING (INTERNET OF THINGS)

by

K.Deepika	2211CS050126
K.Neelaveni	2211CS050132
K.Poojitha	2211CS050136

Under the esteemed guidance of

Mrs. Priyanka Chandragiri
Assistant Professor



Department of Computer Science and Engineering (Internet of Things)

School of Engineering

MALLAREDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana, 500100

2026

Web-Based AI-Secured User Authentication System : SAHAYAA

A Project Report submitted in partial fulfillment of the requirements for the Award of Degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING (INTERNET OF THINGS)

By

K. Deepika	2211CS050126
K. Neelaveni	2211CS050132
K. Poojitha	2211CS050136

Under the esteemed guidance of

Mrs. Priyanka Chandragiri
Assistant Professor



Department of Computer Science and Engineering (Internet of Things)

School of Engineering

MALLAREDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering (Internet of Things)

CERTIFICATE

This is to certify that the project report entitled “**Web-Based AI-Secured User Authentication System : SAHAYAA**”, submitted by **K. Deepika (2211CS050126), K. Neelaveni (2211CS050132), K. Poojitha (2211CS050136)**, towards the partial fulfillment for the award of Bachelor’s Degree in Computer Science and Engineering - Internet of Things from the Department of Internet of Things, Malla Reddy University, Hyderabad, is a record of bonafide work done by him/ her. The results embodied in the work are not submitted to any other University or Institute for award of any degree or diploma.

Internal Guide

Mrs.Priyanka Chandragiri
Associate Professor

Dean(Internet of things)

Dr. G. Anand Kumar
Professor & Dean

External Examiner

DECLARATION

We hereby declare that the project report entitled **“Web-Based AI-Secured User Authentication System : SAHAYAA”** has been carried out by us and this work has been submitted to the **Department of Computer Science and Engineering (Internet of Things), Malla Reddy University, Hyderabad** in partial fulfillment of the requirements for the award of degree of Bachelor of Technology. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place:

Date:

K.Deepika	2211CS050126
K.Neelaveni	2211CS050132
K. Poojitha	2211CS050136

ACKNOWLEDGEMENT

We express our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude to **Dr. V. S. K Reddy, Vice-Chancellor**, for his visionary leadership and unwavering commitment to academic excellence.

We would also like to extend my deepest appreciation to our project guide **Mrs. Priyanka Chandragiri Assistant Professor**, whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We extend our gratitude to **Dr. G. Latha, PRC-convenor**, for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process. We thank our project coordinator **Mr. Ramesh Khanna**, for his timely support.

We are also grateful to **Dr. G. Anand Kumar, Dean (Internet of Things)** for providing us with the necessary resources and facilities to carry out this project

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

K. Deepika	2211CS050126
K. Neelaveni	2211CS050132
K. Poojitha	2211CS050136

ABSTRACT

Sahayaa is a modern web-based donation platform developed to simplify and digitalize the process of contributing to social causes in a secure and transparent manner. The platform connects donors with verified individuals, NGOs, and social campaigns, enabling users to make meaningful contributions with ease. The frontend of the application is built using React.js, providing a responsive, interactive, and user-friendly interface. Features such as campaign browsing, donation tracking, user dashboards, and real time updates enhance the overall user experience. Strapi, a headless CMS, is used as the backend to manage campaigns, users, donation records, and content dynamically through REST APIs. This allows administrators to efficiently create, update, and verify donation campaigns without modifying the frontend code. The platform includes advanced features such as secure user authentication, real-time donation progress tracking, digital donation receipts, and donation history management. Additional enhancements like AI-based donation recommendations, campaign notifications, and feedback systems improve user engagement and transparency. Payment processing is securely integrated to ensure safe transactions. By leveraging React for frontend development, Strapi for backend content management, and modern web technologies, Sahayaa provides a scalable, secure, and efficient donation platform. The system aims to promote social responsibility by making charitable giving accessible, trustworthy, and impactful through technology. The platform incorporates several advanced features such as secure user authentication, real-time donation progress tracking, digital donation receipts, and donation history management. In addition, AI-based donation recommendations, campaign notifications, and feedback systems are integrated to improve user engagement and maintain transparency. Secure payment processing is also implemented to ensure safe and reliable financial transactions. By leveraging modern web technologies like React for frontend development and Strapi for backend management, SAHAYAA offers a scalable, secure, and efficient solution for digital donations.

Keywords - *AI-based Authentication, Behavioral Biometrics, Web Security, Anomaly Detection, Cybersecurity*

INDEX

Table of Content

Chapters	Contents	Page No
Chapter :1	Introduction	01-03
	1.1 Problem Definition & Description	01
	1.2 Objective of the Project	02
	1.3 Scope of the project	03
Chapter :2	System Analysis	04-09
	2.1 Existing System	04-05
	2.1.1 Background & Literature Survey	05
	2.1.2 Limitations of Existing System	05-06
	2.2 Proposed System	06-07
	2.2.1 Advantages of Proposed System	06-07
	2.3 Software & Hardware Requirements	07-08
	2.3.1 Software Requirements	07-08
	2.3.2 Hardware Requirements	08
	2.4 Feasibility Study	08-09
	2.4.1 Technical Feasibility	08-09
	2.4.2 Robustness & Reliability	09
	2.4.3 Economic Feasibility	09
Chapter :3	Architectural Design	10-19
	3.1 Modules Design	10-11
	3.1.1 Module 1: User Management Module	10
	3.1.2 Module 2: Campaign Management Module	10
	3.1.3 Module 3: Donation Processing Module	11
	3.1.4 Module 4: Payment Integration Module	11
	3.1.5 Module 5: Dashboard & Analytics Module	11
	3.2 Methodology & Algorithm Design	12-13
	3.2.1 Method 1	12
	3.2.2 Method 2	12
	3.2.3 Algorithm 1	12
	3.2.4 Algorithm 2	12-13
	3.3 Project Architecture	13-19
	3.3.1 Complete Architecture	13
	3.3.2 Class Diagram	14
	3.3.3 Use Case Diagram	16
	3.3.4 Sequence Diagram	17
	3.3.5 Deployment Diagram	18
	3.3.6 Activity Diagram	19
Chapter :4	Implementation & Testing	20-26
	4.1 Coding Blocks	20
	4.1.1 Main Function Block	20
	4.1.2 Function Block 1	22-26

	4.2 Test Case Study	25-26
	4.2.1 Test Case 1	25
	4.2.2 Test Case 2	26
	4.2.3 Test Case 3	26
Chapter :5	Results	27-33
	5.1 Resulting Screens	27-30
	5.2 Resulting Table	31
	5.3 Resulting Graph	31-32
	5.4 Results Analysis	33
Chapter :6	Conclusions & Future Scope	34
	6.1 Conclusions	35
	6.2 Future Scope	36

BIBLIOGRAPHY

PAPER PUBLICATION

POSTER

List of Diagrams

Figure No	Description	Page No
3.3.1	Architectural Diagram	13
3.3.2	Class Diagram	14
3.3.3	Use Case Diagram	16
3.3.4	Sequence Diagram	17
3.3.5	Deployment Diagram	18
3.3.6	Active Diagram	19

List of Screenshots

Screenshot No	Description	Page No
5.1.1	Login Page	28
5.1.2	Home Page	28
5.1.3	Register Page	29
5.1.4	Helping Page	29
5.1.5	Donate Page	30
5.1.6	Complete Your Donation	30

List of Graphs

Graph No	Description	Page no
5.2.1	Total Donation time	31
5.2.2	Goal Process	32
5.2.3	Donation Trends	33

List of Tables

Table No	Description	Page no
4.2.1	Test Case 1	25
4.2.2	Test Case 2	26
4.2.3	Test Case 3	26
5.3.1	Resulting Table	31

CHAPTER - 1

INTRODUCTION

1.1 Problem definition & description

With the rapid growth of digital platforms and web applications, secure user authentication has become a critical requirement. Traditional authentication systems, such as username and password mechanisms, are highly vulnerable to cyber threats including password theft, phishing attacks, brute-force attacks, and identity fraud. Many existing web-based systems rely on static credentials, which can easily be compromised if proper security mechanisms are not implemented. As a result, users' personal data, financial information, and sensitive records are at significant risk of unauthorized access.

Another major problem with conventional authentication methods is the lack of intelligent monitoring and adaptive security mechanisms. Most systems do not analyze user behavior patterns or detect suspicious login attempts in real time. Attackers can exploit these weaknesses to gain unauthorized entry into systems without triggering any alerts. This creates a serious security challenge for organizations and individuals who depend on web applications for communication, transactions, and data storage.

Furthermore, users often struggle with managing multiple passwords across different platforms, which leads to poor password practices such as using weak passwords or repeating the same password on multiple websites. This significantly increases the chances of credential leakage and account compromise. Additionally, traditional systems provide limited protection against automated bot attacks and credential stuffing, which are becoming increasingly common in modern cyber environments.

To address these issues, a **Web-Based AI-Secured User Authentication System named SAHAYAA** is proposed. This system integrates Artificial Intelligence techniques to enhance authentication security by detecting abnormal login behavior, preventing unauthorized access, and strengthening identity verification processes. The SAHAYAA system aims to provide a more reliable, intelligent, and user-friendly authentication framework that improves security while maintaining ease of access for legitimate users. By leveraging AI-based analysis and advanced security protocols, the system helps protect user accounts and sensitive data from evolving cyber threats.

1.2 Aim and objectives of the project

The primary objective of the **Web-Based AI-Secured User Authentication System: SAHAYAA** is to develop a secure and intelligent authentication framework that protects user accounts from unauthorized access and cyber threats. The system aims to enhance the traditional login process by integrating Artificial Intelligence techniques that can identify suspicious activities and strengthen the overall security of web-based platforms. By improving authentication mechanisms, the project seeks to ensure that only legitimate users are able to access the system and its services.

Another important objective of the project is to provide a reliable and user-friendly authentication process that minimizes the risks associated with weak passwords and credential theft. The system focuses on implementing advanced security measures such as intelligent verification, secure login monitoring, and protection against common cyber attacks like phishing, brute-force attacks, and credential stuffing. This helps in maintaining the confidentiality and integrity of user data.

The project also aims to improve system efficiency by analyzing user login behavior and detecting anomalies in real time. Through AI-based analysis, the system can recognize unusual login attempts and take preventive actions to block or verify suspicious activities. This proactive approach strengthens the security layer and reduces the chances of successful cyber intrusions.

Finally, the SAHAYAA system is designed to provide a scalable and adaptable authentication solution that can be integrated into various web applications and platforms. The project seeks to create a modern security model that not only protects sensitive information but also enhances user trust and reliability in digital systems. By combining AI technology with web-based authentication, the project contributes to building safer and more secure online environment

By combining AI technology with web-based authentication, the project contributes to building safer and more secure online environment

1.3 Scope of the project

The scope of the **Web-Based AI-Secured User Authentication System: SAHAYAA** focuses on developing a secure and intelligent web-based platform that enhances the traditional user authentication process using Artificial Intelligence techniques. The project covers the design and implementation of a system that allows users to register, log in, and access services through a protected authentication mechanism. It includes features such as secure user registration, encrypted password storage, AI-based monitoring of login activities, and detection of suspicious or abnormal access attempts.

The system aims to protect sensitive user information by preventing common cyber threats such as unauthorized access, brute-force attacks, and credential misuse. Additionally, the project focuses on creating a user-friendly interface that ensures easy interaction while maintaining high security standards. The developed system can be integrated with different web applications and platforms, making it adaptable for various organizations and services that require secure user authentication.

However, the scope is limited to web-based environments and focuses mainly on improving authentication security rather than covering all aspects of cybersecurity infrastructure. It includes features such as secure user registration, encrypted password storage, AI-based monitoring of login activities, and detection of suspicious or abnormal access attempts. The system aims to protect sensitive user information by preventing common cyber threats such as unauthorized access, brute- force attacks, and credential misuse.

Additionally, the project focuses on creating a user-friendly interface that ensures easy interaction while maintaining high security standards. It includes features such as secure user registration, encrypted password storage, AI-based monitoring of login activities, and detection of suspicious or abnormal access attempts.

The system aims to protect sensitive user information by preventing common cyber threats such as unauthorized access, brute-force attacks, and credential misuse. Additionally, the project focuses on creating a user-friendly interface that ensures easy interaction while maintaining high security standards.

CHAPTER - 2

SYSTEM ANALYSIS

System analysis is an important phase in the development of the **Web-Based AI-Secured User Authentication System: SAHAYAA**, as it helps in understanding the requirements, challenges, and functionalities needed for the system to operate effectively. In this stage, the existing authentication methods are studied to identify their limitations, security risks, and inefficiencies. Traditional systems mainly rely on basic username and password authentication, which is highly vulnerable to cyber threats such as phishing attacks, brute-force attempts, and unauthorized access. These weaknesses highlight the need for a more secure and intelligent authentication mechanism that can protect user data and prevent security breaches.

2.1 Existing system analysis

The existing user authentication systems used in most web applications mainly rely on traditional methods such as username and password verification to allow users to access services. Although this method is simple and widely implemented, it has several security weaknesses. Password-based authentication is vulnerable to cyber threats such as phishing attacks, brute-force attacks, password guessing, and credential theft. Many users often create weak passwords or reuse the same passwords across multiple platforms, which increases the risk of unauthorized access. Additionally, traditional systems lack intelligent mechanisms to monitor and analyze login activities or detect suspicious behavior.

They usually do not identify unusual login attempts, repeated failed logins, or access from unfamiliar devices or locations. Because these systems depend only on static credentials and do not use advanced technologies like Artificial Intelligence or behavioral analysis, they are less capable of preventing modern cyber attacks. As a result, existing authentication systems provide only basic security and are not fully effective in protecting sensitive user data from evolving cybersecurity threats.

The system is developed as a web application, allowing users to register, log in, and securely access services from any device with internet connectivity. It ensures that only authenticated and verified users can interact with the platform, thereby maintaining trust and data integrity.

2.1.1 Background and literature survey

User authentication is one of the most critical components in modern web-based systems, as it ensures that only authorized users can access digital services and sensitive information. Traditionally, authentication systems relied mainly on simple username and password mechanisms. Although this method has been widely used for many years, it has several security limitations. With the rapid growth of internet usage and online services such as banking, education portals, e-commerce platforms, and cloud-based applications, the need for stronger and more reliable authentication systems has become increasingly important. Cyber attackers continuously develop new techniques such as phishing, brute-force attacks, and credential stuffing to compromise user accounts, making traditional authentication methods less effective.

Several studies in the field of cybersecurity have highlighted the weaknesses of password-based authentication systems. Researchers have suggested that users often create weak passwords or reuse the same passwords across multiple platforms, which significantly increases the risk of data breaches. To overcome these challenges, many researchers have proposed advanced authentication methods such as multi-factor authentication (MFA), biometric verification, and one-time password (OTP) systems. These methods add additional layers of security to the authentication process and reduce the chances of unauthorized access.

In recent years, Artificial Intelligence and machine learning technologies have been introduced into authentication systems to improve security and efficiency. AI-based authentication systems can analyze user behavior patterns such as login time, device information, typing speed, and location data to identify suspicious activities. If any unusual behavior is detected, the system can trigger additional verification steps or temporarily block the login attempt. This intelligent approach helps in preventing many common cyber attacks before they cause damage.

2.1.2 limitations of existing system

1. Weak Password Security:

Most existing authentication systems depend on username and password mechanisms. Users often create weak passwords or reuse the same passwords across multiple platforms, making it easier for attackers to guess or steal credentials.

2. Vulnerability to Cyber Attacks:

Traditional systems are highly vulnerable to cyber threats such as phishing attacks, brute-force

attacks, and credential stuffing. These attacks can allow unauthorized users to gain access to sensitive information.

3. Lack of Intelligent Monitoring:

Existing systems do not include intelligent monitoring mechanisms to analyze login activities. They fail to detect unusual login attempts or suspicious behavior patterns during the authentication process.

4. No Behavioral Analysis:

Most traditional authentication systems do not analyze user behavior such as login location, device information, or login timing. Without this analysis, it becomes difficult to identify abnormal activities or potential threats.

5. Limited Security Features:

Many existing systems rely only on static credentials and do not use advanced technologies such as Artificial Intelligence or machine learning to enhance security and adapt to new cyber threats.

6. Lack of Real-Time Alerts:

Traditional authentication systems often do not provide real-time alerts or additional verification steps when suspicious login attempts occur, which increases the chances of unauthorized access, which significantly increases the risk of data breaches. To overcome these challenges, many researchers have proposed advanced authentication methods such as multi-factor authentication (MFA).

2.2 Proposed system analysis

The proposed system, **SAHAYAA**, is a web-based AI-secured user authentication framework designed to overcome the limitations of traditional authentication methods. The system integrates artificial intelligence techniques to analyze user behavior, login patterns, and contextual information during the authentication process. SAHAYAA employs intelligent algorithms to detect anomalies and suspicious activities in real time, enabling dynamic decision-making for user verification.

2.2.1 Advantages of proposed system

- **Enhanced Security:**

By integrating Artificial Intelligence, the system can monitor login patterns, detect unusual

activities, and prevent unauthorized access, reducing the risk of cyber attacks such as phishing, brute-force attacks, and credential theft.

2. Intelligent User Verification:

The AI component analyzes user behavior, such as login time, device type, and location, to identify suspicious activities. This allows the system to take proactive security measures, such as triggering additional verification steps.

3. Protection of Sensitive Data:

With encrypted credential storage and real-time monitoring, the system ensures that users' personal and sensitive information is safeguarded against cyber threats.

4. User-Friendly Interface:

The proposed system is designed with a simple and intuitive interface, making the authentication process easy for users while maintaining high levels of security.

5. Real-Time Threat Detection:

The AI-driven system can instantly identify abnormal login attempts or patterns, allowing immediate action to block potential intruders and notify users of suspicious activity.

6. Scalability and Adaptability:

SAHAYAA can be integrated with multiple web applications and platforms, making it suitable for various industries such as education, banking, e-commerce, and corporate systems.

7. Reduced Dependence on Passwords Alone:

The combination of AI-based monitoring and advanced verification reduces reliance solely on passwords, minimizing risks associated with weak or reused passwords.

2.3 Hardware and software requirements

For the successful development and deployment of the SAHAYAA Web-Based AI-Secured User Authentication System, the following hardware and software requirements are identified:

2.3.1 Software requirements

- **Operating System:** Windows 10/11, Linux, or macOS
- **Frontend:** React.js, HTML, CSS, JavaScript
- **Backend:** Strapi (headless CMS) with REST APIs
- **Database:** PostgreSQL, MySQL, or MongoDB
- **AI/ML:** TensorFlow, Keras, or scikit-learn for donation recommendations

- **Authentication & Security:** JWT/OAuth, password encryption, HTTPS, secure payment
- **Development Tools:** Visual Studio Code, Git/GitHub

2.3.2 Hardware requirements

- **Processor:** Intel i3 or higher (Quad-core recommended)
- **RAM:** Minimum 8 GB (16 GB recommended for AI processing)
- **Storage:** Minimum 500 GB HDD/SSD (SSD preferred for faster performance)
- **Graphics:** Dedicated GPU (Optional, required for heavy AI model processing)
- **Networking:** Stable internet connection for web access and cloud integration
- **Input Devices:** Keyboard and mouse for user interaction
- **Display:** Minimum 15-inch monitor with 1366×768 resolution

2.4 Feasibility study

The feasibility study of the SAHAYAA Web-Based AI-Secured User Authentication System indicates that the project is practical and achievable. Technically, it can be implemented using widely available web technologies and AI libraries for behavior analysis and anomaly detection. Operationally, the system is user-friendly and enhances security without disrupting existing workflows, making it suitable for organizations and end-users. Economically, the use of open-source software and existing hardware resources keeps development costs low, while the system's ability to prevent cyber threats can save significant financial and reputational losses. Overall, the project is feasible from technical, operational, and economic perspectives.

2.4.1 Technical feasibility

The technical feasibility of the SAHAYAA Web-Based AI-Secured User Authentication System evaluates whether the necessary technology, tools, and resources are available to AI and machine learning functionalities for analyzing user behavior and detecting anomalies can be implemented using open-source libraries like TensorFlow, Keras, and Scikit-learn. The required hardware, including standard processors, RAM, and storage, is sufficient to run the system efficiently. Additionally, secure database management systems like MySQL or PostgreSQL provide robust data handling and encryption support. Since all the necessary technologies are readily available and compatible, the project is technically feasible and can be successfully executed within the existing technological infrastructure.

successfully develop and implement the system. The system relies on widely used web development technologies such as HTML, CSS, JavaScript for the front-end, and backend frameworks like Python Django or Node.js, which are well-supported and reliable. AI and machine learning functionalities for analyzing user behavior and detecting anomalies can be implemented using open-source libraries like TensorFlow, Keras, and Scikit-learn. The required hardware, including standard processors, RAM, and storage, is sufficient to run the system efficiently. Additionally, secure database management systems like MySQL or PostgreSQL provide robust data handling and encryption support. Since all the necessary technologies are readily available and compatible, the project is technically feasible and can be successfully executed within the existing technological infrastructure.

2.4.2 Robustness & reliability

The **robustness and reliability** of the SAHAYAA Web-Based AI-Secured User Authentication System ensure that it can consistently perform its functions under various conditions while maintaining high security standards. The system is designed to handle multiple concurrent users and simultaneous login requests without performance degradation, making it robust for real-world web applications. AI-based monitoring continuously analyzes user behavior to detect anomalies, providing reliable protection against unauthorized access and cyber threats. Data encryption and secure storage mechanisms further enhance system reliability by safeguarding sensitive user information. Additionally, the system is capable of recovering gracefully from unexpected errors or failures, minimizing downtime and ensuring uninterrupted service. Overall, the combination of intelligent monitoring, secure data handling, and error-resilient architecture makes SAHAYAA a robust and dependable authentication solution.

2.4.3 Economical feasibility

The **economical feasibility** of the SAHAYAA Web-Based AI-Secured User Authentication System assesses whether the project can be developed and maintained within a reasonable budget while providing significant benefits. The system primarily uses open-source technologies such as Python, Django/Flask, HTML, CSS, JavaScript, and AI libraries like TensorFlow and Keras, which minimizes software licensing costs. Existing hardware infrastructure, such as standard computers and servers, is sufficient, reducing the need for additional investment. Implementing this system can prevent costly data breaches, account compromises, and financial losses associated with cyber attacks, making it a cost-effective solution for organizations.

CHAPTER - 3

ARCHITECTURAL DESIGN

3.1 Modules design

The SAHAYAA Web-Based AI-Secured User Authentication System is organized into several key modules to ensure secure and efficient authentication. The User Registration Module allows new users to create accounts with encrypted credentials, while the Login and Authentication Module verifies user credentials and uses AI to detect abnormal login patterns. The AI-Based Monitoring Module continuously analyzes user behavior, triggering alerts or additional verification for suspicious activities. The Password Management Module enables secure password updates and resets, and the Admin Control Module allows administrators to monitor system activity, manage users, and generate reports. Finally, the Security and Encryption Module ensures safe data storage and transmission using encryption and SSL/TLS protocols. Together, these modules create a robust, intelligent, and reliable authentication system.

3.1.1 User Management Module

The User Management Module is responsible for handling all user-related activities within the system. It provides functionalities such as user registration, login, logout, and profile management. Users can create accounts by providing basic details such as name, email, and password. This module ensures secure authentication using proper validation techniques and encryption methods to protect user credentials.

3.1.2 Campaign Management Module

The Campaign Management Module is one of the core components of the system. It allows administrators to create, manage, and monitor donation campaigns efficiently. Campaigns may include social causes such as education support, medical assistance, disaster relief, and community development.

The module also includes a verification process to ensure that all campaigns are genuine and trustworthy before being displayed to users.

Additionally, administrators can edit or delete campaigns, track their performance, and update campaign progress. This module plays a key role in maintaining transparency and trust within the platform.

3.1.3 Donation Processing Module

The Donation Processing Module handles the complete donation workflow. It allows users to browse available campaigns and contribute funds to the causes they wish to support.

Once a user selects a campaign, the module processes the donation request by capturing the donation amount and user details.

It ensures that transactions are properly recorded in the system. After successful donation, the system generates a digital receipt for the donor as proof of contribution.

This module also maintains donation history, allowing users to track all their past contributions. It ensures accuracy, reliability, and transparency in handling donations.

3.1.4 Payment Integration Module

The Payment Integration Module is responsible for handling secure financial transactions. It integrates with trusted payment gateways to enable online payments through various methods such as debit cards, credit cards, UPI, and net banking.

This module ensures data security by implementing encryption and secure communication protocols. It verifies transaction status and handles success or failure responses appropriately.

In case of failed transactions, proper error handling and user notifications are provided. This module is critical for ensuring user trust and maintaining the financial integrity of the platform.

3.1.5 Dashboard & Analytics Module

The Dashboard and Analytics Module provides a visual representation of system data for both users and administrators. It includes interactive dashboards that display information such as total donations, number of active campaigns, and donation progress.

Graphs and charts such as line graphs, bar charts, and progress indicators are used to present data in an easy-to-understand format.

Users can view their personal donation statistics, while administrators can analyze overall system performance.

This module helps in decision-making, monitoring campaign success, and improving user engagement through data-driven insights.

The analytics component uses intelligent data processing techniques to track user behavior patterns and detect anomalies.

3.2 Methodology & algorithm design

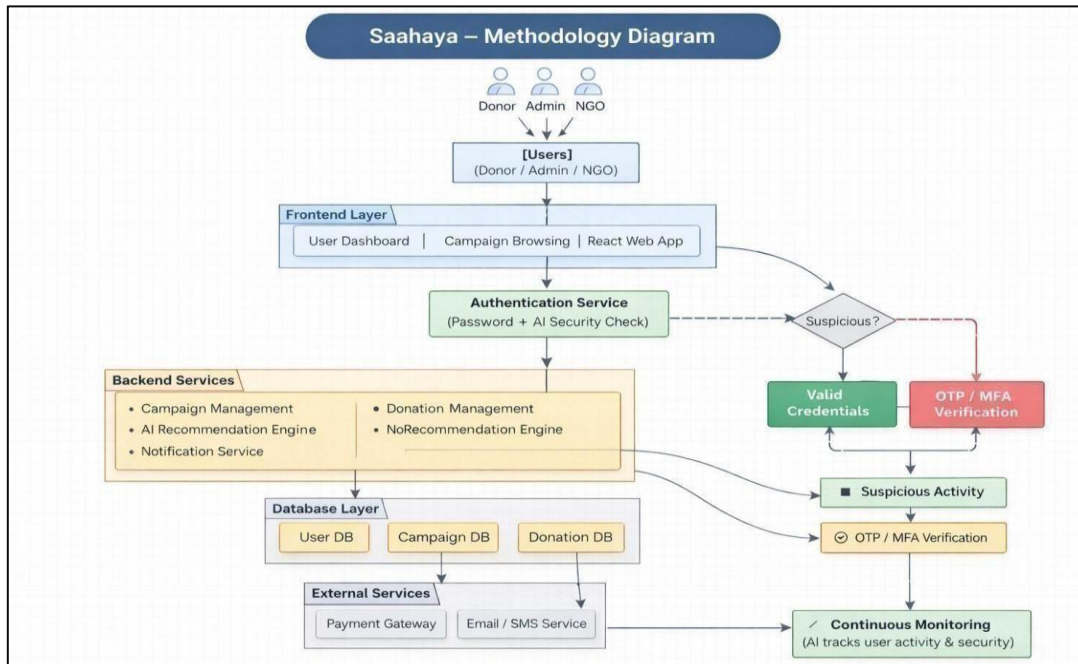


Figure3.2 Methodology of Web-Based AI-Secured User Authentication System : SAHAYAA

3.2.1 Authentication Method

The diagram illustrates the overall architecture and interaction between different components of the system, highlighting the flow of data and responsibilities across the frontend, backend, database, and external services.

3.2.2 Modular Development Method

On the **frontend**, users, admins, and donors interact with various interfaces such as the User Dashboard, Campaign Browsing, React Web Application, and Donation Tracking. These components provide the user experience layer where inputs are collected and results are displayed.

3.2.3 Notification System Algorithm

The **backend** handles core business logic and services. It includes modules like the Strapi CMS, Authentication Service, AI Recommendation Engine, Donation Management, Campaign Management, and Notification Service. These modules manage user authentication, campaign operations, donation processing, and sending notifications to users.

3.2.4 Donation Processing Algorithm

The system relies on several **databases** to store information securely. These include the Donation DB for storing donation records, Campaign DB for campaign details, and User DB for managing

Integration with **external services** such as Email/SMS services and Payment Gateways supports communication with users and processes financial transactions securely.

3.3 Project architecture

The architecture of the **SAHAYAA Web-Based AI-Secured User Authentication System** is designed as a multi-layered framework that integrates frontend interfaces, backend services, databases, and external systems to deliver a secure and intelligent authentication experience.

At the **frontend layer**, users interact through a responsive web application built with modern technologies like React, providing interfaces such as user dashboards, login forms, and campaign browsing. This layer collects user inputs and displays relevant information, ensuring a smooth and user-friendly experience.

The **backend layer** acts as the system’s core, managing business logic and security. It includes services for user authentication, AI-powered behavior analysis, campaign and donation management, and notification delivery. The backend authenticates users by verifying credentials, analyzing login patterns with AI to detect anomalies, and enforcing additional security measures when required.

3.3.1 Architecture diagram

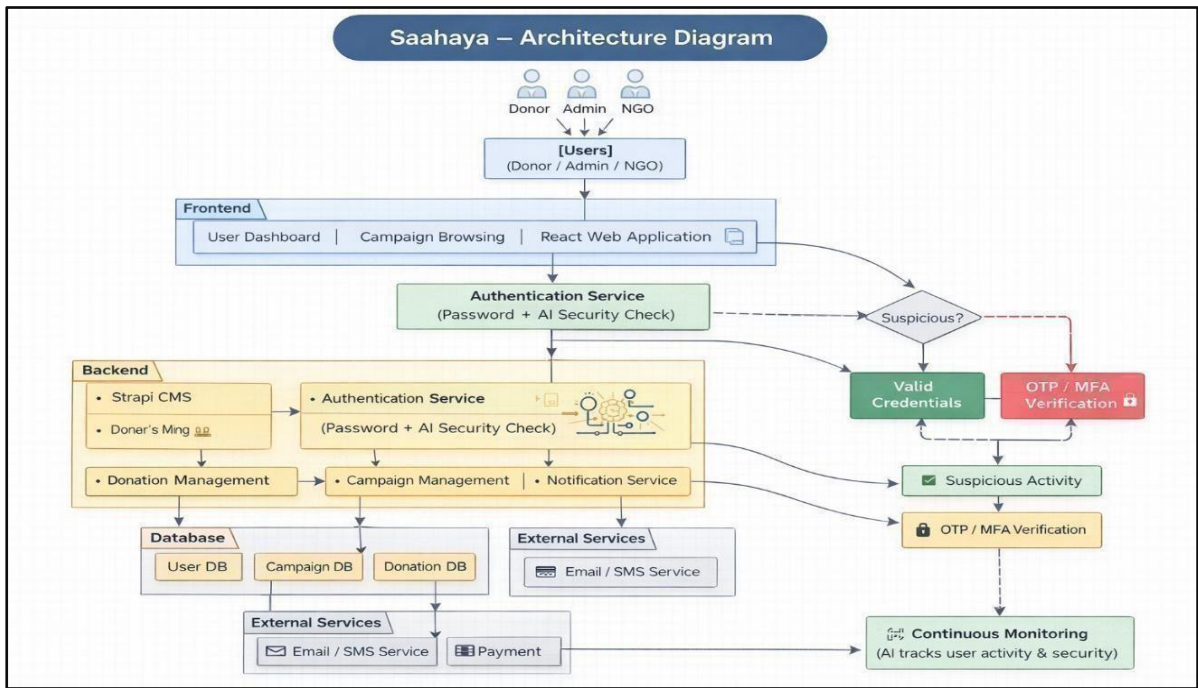


Figure3.3.1 Architecture Diagram of Web-Based AI-Secured User Authentication System : SAHAYAA

The diagram represents a **high-level architecture of a donation platform** that connects three main types of users—donors, administrators, and NGOs—through a web-based system. All users interact with the system via a React-based frontend application, which provides key features such as a user dashboard, campaign browsing, and donation tracking. This frontend acts as the interface through which users perform actions like exploring campaigns, managing accounts, and making donations.

Behind the frontend lies a structured backend composed of several specialized services. The system uses a content management system (Strapi CMS) to manage and organize campaign-related content. An authentication service handles user login, registration, and role-based access for donors, admins, and NGOs. Additionally, an AI recommendation engine analyzes user behavior and preferences to suggest relevant campaigns, improving user engagement. Core functionalities are managed by the donation management service, which processes donations, and the campaign management service, which allows NGOs and admins to create, update, and manage campaigns.

3.3.2 Class diagram

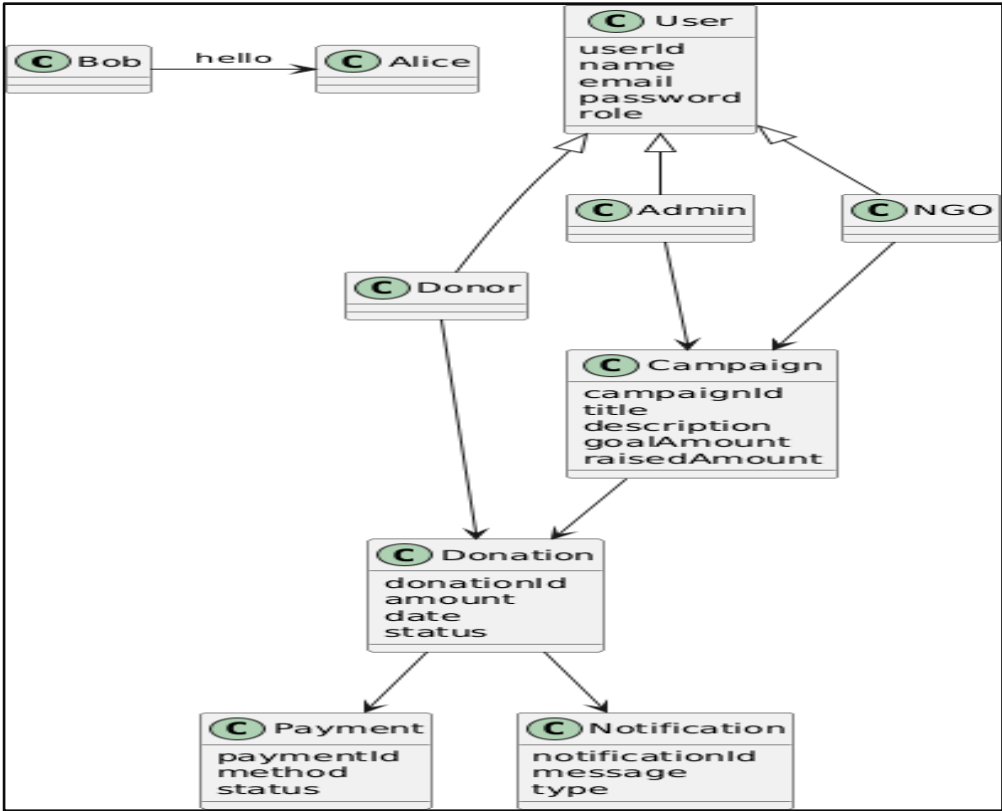


Figure 3.3.2 Class Diagram of Web-Based AI-Secured User Authentication System : SAHAYAA

1. User Access & Authentication

- User(Donor/ Admin/ NGO) opens the **React Web Application**
- Enters login/signup details
- → Sent to **Authentication Service**
- →Validated with**User DB**
- →Access granted (Dashboard loads)

2. Campaign Browsing& Recommendation

- User navigates to **Campaign Browsing**
- → Request sent to **Campaign Management**
- →Fetches campaigns from **Campaign DB**
- →(Optional) AI Recommendation Engine suggests personalized campaigns
- → Display campaigns on UI

3. Campaign Creation (Admin/ NGO)

- Admin/NGO submits campaign details via UI
- → Sent to **Campaign Management**
- →Validates and stores in **Campaign DB**
- → Confirmation sent back to UI

4. Donation Process

- Donor selects campaign and clicks “Donate”
- → Request goes to **Donation Management**
- → Donation details processed
- → Payment request sent to **Payment Gateway**

5. Payment Processing

- Payment Gateway processes transaction
- → Sends success/failure response to **Donation Management**

6. Donation Recording

- On success:
- Donation stored in **Donation DB**

- Campaign amount updated in **Campaign DB**
- On failure:
- Error returned to user

7. Notification Process

- DonationManagement triggers **Notification Service**
- →Notification sent via **Email/SMS Service**
- →User receives confirmation

8. Donation Tracking

- User accesses **Donation Tracking Dashboard**
- →Fetches data from **Donation DB**
- →Displays donation history/status

3.3.3 Use case diagram

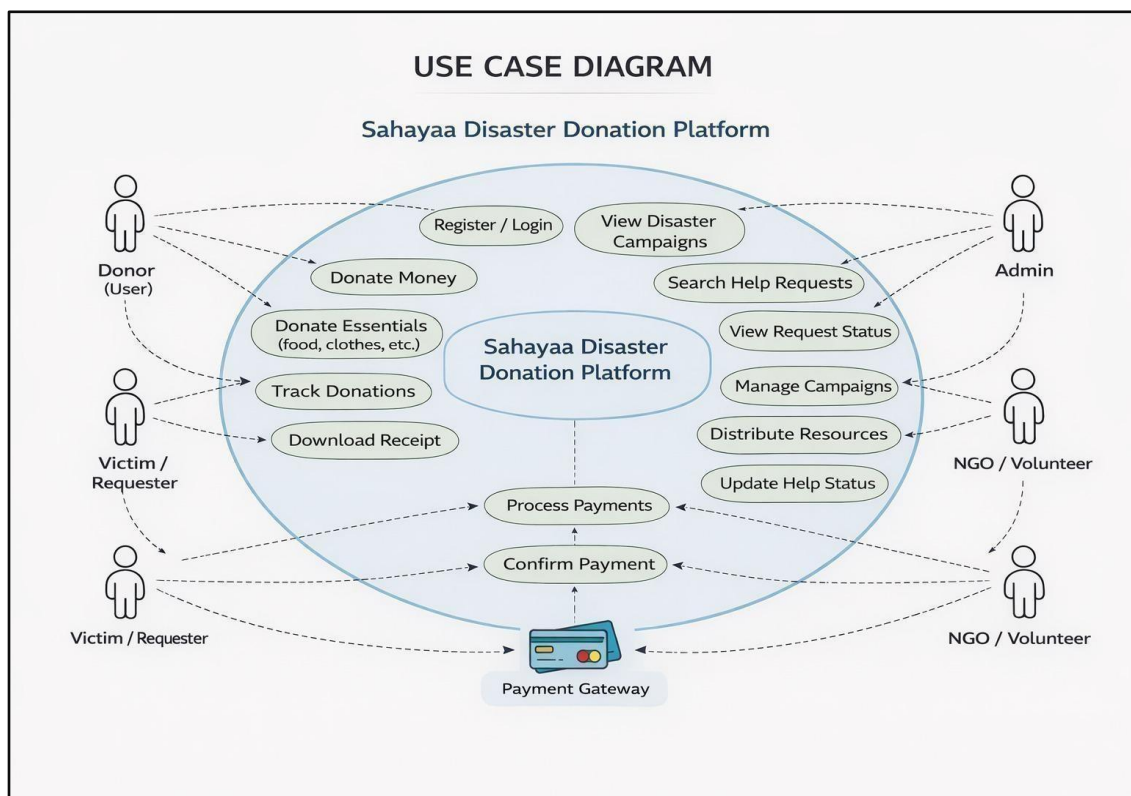


Figure 3.3.3 Use Case Diagram of Web-Based AI-Secured User Authentication System : SAHAYAA

The class diagram represents the structure of the donation platform by showing the main classes, their attributes, and relationships. The central class is **User**, which is further specialized into **Donor**, **Admin**, and **NGO**, representing different roles in the system.

The **Campaign** class contains details about fundraising campaigns such as title, description, goal amount, and current status. Admins and NGOs are responsible for creating and managing campaigns.

The **Donation** class represents contributions made by donors. Each donation is associated with a specific campaign and includes details like amount, date, and status. The **Payment** class handles the transaction process for each donation.

3.3.4 Sequence diagram

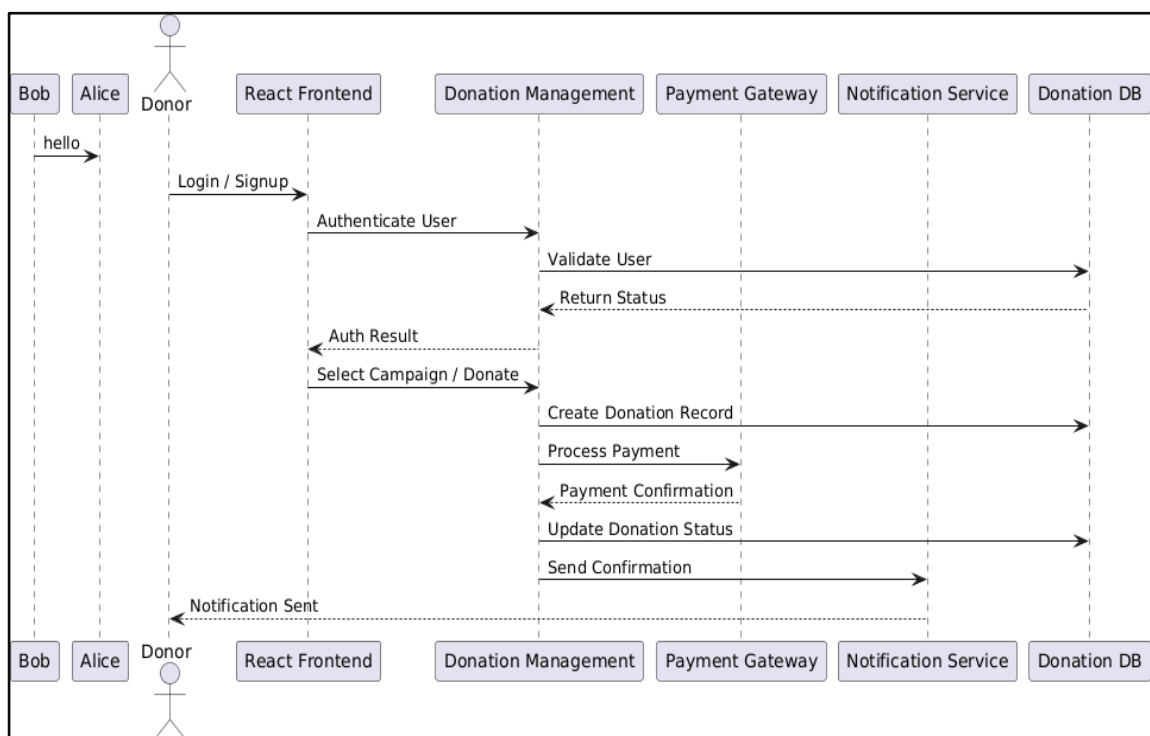


Figure 3.3.4 Sequence Diagram of Web-Based AI-Secured User Authentication System : SAHAYAA

When a donor logs in, the frontend sends authentication requests to the backend, which validates the user through the database. After successful login, the donor can browse campaigns and make a donation. The backend creates a donation record, processes the payment through the payment gateway, updates the database, and triggers notifications to the donor

The Use Case Diagram of the Sahayaa system illustrates the interactions between different users and the platform. The main actors in the system are **Donor**, **Admin**, and **NGO**, each having specific roles and functionalities.

The **Donor** can register or log in, browse available campaigns, make donations, track donation history, and receive notifications. The **NGO** is responsible for creating and updating fundraising campaigns, while the **Admin** manages users, verifies campaigns, and monitors system activities.

3.3.5 Deployment diagram

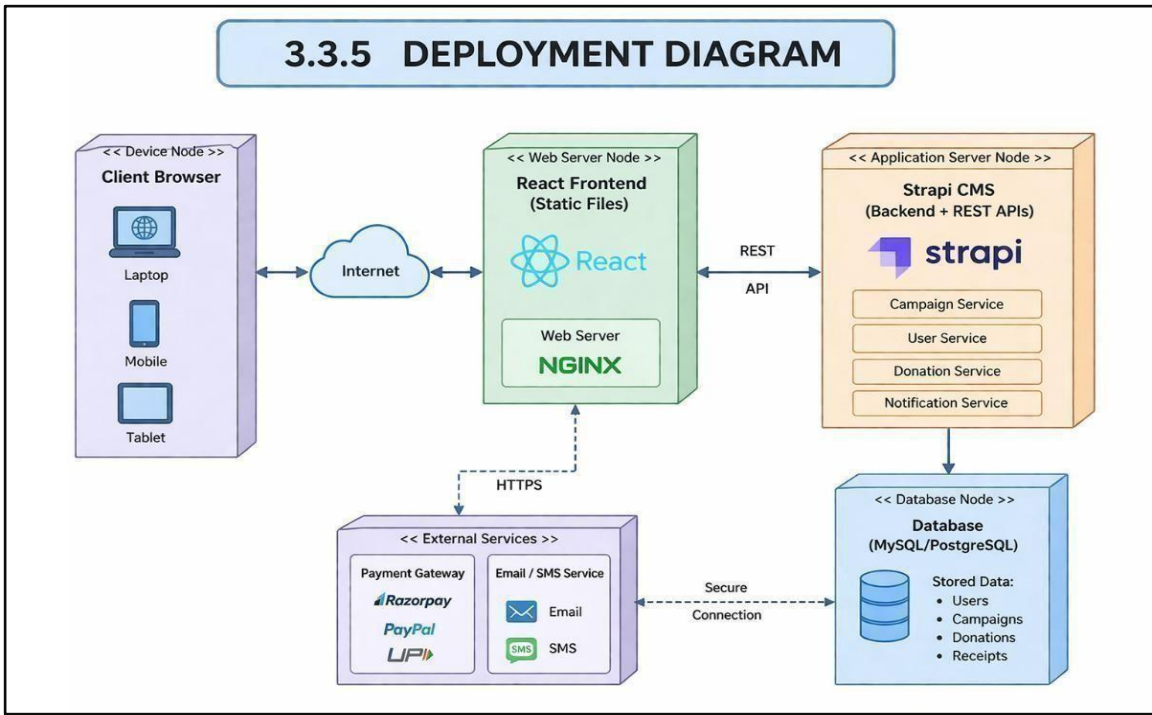


Figure 3.3.5 Deployment Diagram of Web-Based AI-Secured User Authentication System : SAHAYAA

The sequence diagram for the Sahayaa donation platform illustrates the step-by-step interaction between the Donor, Frontend (React App), Backend (Donation Management), Payment Gateway, Notification Service, and Donation Database.

When a donor logs in, the frontend sends authentication requests to the backend, which validates the user through the database. After successful login, the donor can browse campaigns and make a donation. The backend creates a donation record, processes the payment through the payment gateway, updates the database, and triggers notifications to the donor.

3.3.6 Activity diagram

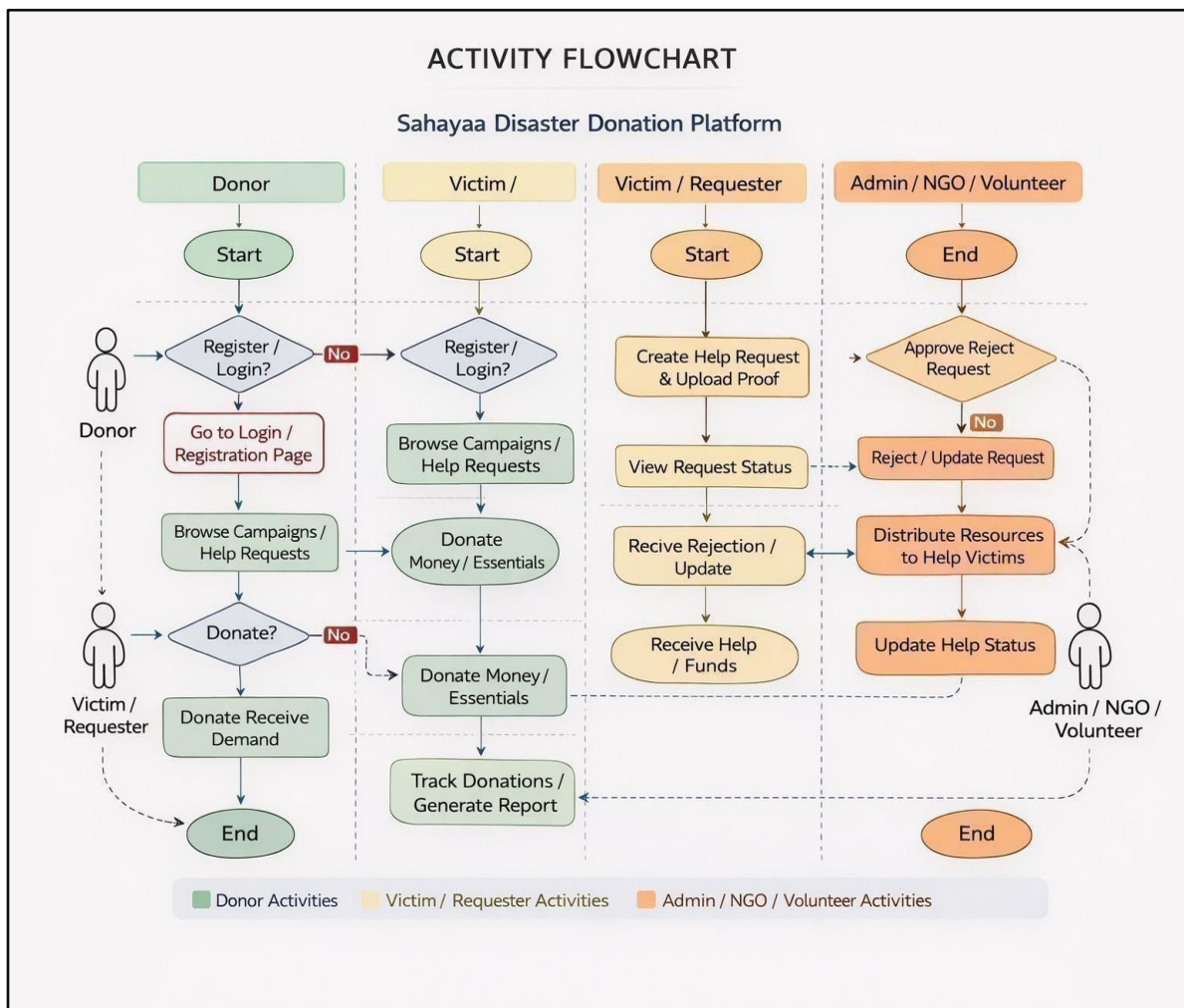


Figure 3.2.6.7 Activity Diagram of Web-Based AI-Secured User Authentication System : SAHAYAA

The Activity Diagram of the Sahayaa donation platform illustrates the complete workflow of the system, representing how different processes are carried out from the moment a user enters the platform until the completion of a donation.

The process begins when a user, such as a donor, administrator, or NGO representative, accesses the system through a web browser or mobile device. The user is first required to register or log in by providing valid credentials, after which the system performs authentication by verifying the entered details with the stored database records.

In the SAHAYAA system, the activity diagram begins with the user accessing the web application. The user can either register as a new user or log in using existing credentials. During registration, the system collects use details, validates the input, and securely stores the information.

CHAPTER - 4

IMPLEMENTITION & TESTING

Sahayaa is implemented as a modern web-based donation platform that connects donors with verified NGOs, social campaigns, and individuals. The **frontend is developed using React.js**, providing a responsive and user-friendly interface for browsing campaigns, tracking donations, and viewing dashboards.

The **backend is built using Strapi**, a headless CMS, which manages users, campaigns, and donation records dynamically through REST APIs. This allows admins and NGOs to efficiently create, update, and verify campaigns without changing frontend code.

Key features implemented include **secure authentication, real-time donation tracking, digital receipts, AI-based campaign recommendations, notifications, and integrated payment processing**. The system ensures **security, transparency, and scalability**, making charitable contributions easy, trustworthy, and impactful.

4.1 Coding blocks

4.1.1 Main functional block

```
#root {  
  max-width: 1280px;  
  margin: 0 auto;  
  padding: 2rem;  
  text-align: center;  
}  
  
.logo {  
  height: 6em;  
  padding: 1.5em;  
  will-change: filter;  
  transition: filter 300ms;  
}  
.logo: hover {  
  filter: drop-shadow(0 0 2em #646cfaa);  
}  
.logo.react: hover {  
  filter: drop-shadow(0 0 2em #61dafbaa);  
}
```

```

to {
  transform: rotate(360deg);
}
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}

.read-the-docs {
  color: #888;
}

.help-card {
  transition: all 0.3s ease;
}

.help-card:hover {
  transform: translateY(-8px);
  box-shadow: 0 15px 30px rgba(0,0,0,0.15);
}

import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import Home from "../pages/Home";
import Navbar from "../components/home/Navbar";
import Login from "../pages/Login";
import Register from "../pages/Register";
import Donate from "../pages/Donate";
import SmallHelps from "../pages/SmallHelps";
import VolunteerOpportunities from "../components/smallhelps/VolunteerOpportunities";
import DisasterDetails from "../components/donate/DisasterDetails";
import RequestHelp from "../pages/RequestHelp";
import HelpRequests from "../pages/HelpRequests";
import HelpRequestDetails from "../pages/HelpRequestDetails";
import HowItWorks from "../components/home/HowItWorks";

function App() {
  return (
    <Router>

```

```

    { /*Navbar stays on top for all pages */}
    <Navbar />

    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/donate" element={<Donate />} />
      <Route path="/login" element={<Login />} />
      { /*Navbar stays on top for all pages */}
    <Navbar />

    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/donate" element={<Donate />} />
      <Route path="/login" element={<Login />} />
      <Route path="/request-help" element={<RequestHelp />} />
      <Route path="/help-requests" element={<HelpRequests />} />
      <Route path="/disaster/:id" element={<DisasterDetails />} />
      <Route path="/help-request/:id" element={<HelpRequestDetails />} />
      <Route path="/how-it-works" element={<HowItWorks />} />
    </Routes>
  </Router>
  "node_modules/@ai-sdk/provider": {
    "version": "2.0.0",
    "resolved": "https://registry.npmjs.org/@ai-sdk/provider/-/provider-2.0.0.tgz",
    "integrity": "sha512-6o7Y2SeO9vFKB8lArHXehNuusnpddKPk7xqL7T2/b+OvXMRIXUO1rR4wcv1hAFUAT9avGZshty3Wlua/XA7TvA==",
    "license": "Apache-2.0",
    "dependencies": {
      "json-schema": "^0.4.0"
    },
    "engines": {
      "node": ">=18"
    }
  },
  "license": "Apache-2.0",
  "dependencies": {
    "json-schema": "^0.4.0"
  },
  "engines": {
    "node": ">=18"
  }

```



```

    <Route path="/register" element={<Register />} />
    <Route path="/small-helps" element={<SmallHelps />} />
    <Route
      path="/volunteer-opportunities"
      element={<VolunteerOpportunities />}
    />
    <Route path="/request-help" element={<RequestHelp />} />
    <Route path="/help-requests" element={<HelpRequests />} />
    <Route path="/disaster/:id" element={<DisasterDetails />} />
    <Route path="/help-request/:id" element={<HelpRequestDetails />} />
    <Route path="/how-it-works" element={<HowItWorks />} />
  </Routes>
</Router>
);
}

export default App;

```

4.1.2 Functional block 1

```

{
  "name": "sahayaa-backend",
  "version": "0.1.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "sahayaa-backend",
      "version": "0.1.0",
      "dependencies": {
        "@strapi/plugin-cloud": "5.33.2",
        "@strapi/plugin-users-permissions": "5.33.2",
        "@strapi/strapi": "5.33.2",
        "better-sqlite3": "12.4.1",
        "fs-extra": "^10.0.0",
        "mime-types": "^2.1.27",
        "razorpay": "^2.9.6",
        "react": "^18.0.0",
        "react-dom": "^18.0.0",

```

```

    },
    "engines": {
      "node": ">=18"
    },
    "peerDependencies": {
      "react": "^18 || ^19 || ^19.0.0-rc",
      "zod": "^3.25.76 || ^4"
    },
    "peerDependenciesMeta": {
      "zod": {
        "optional": true
      }
    }
  },
  "node_modules/@babel/code-frame": {
    "version": "7.27.1",
    "resolved": "https://registry.npmjs.org/@babel/code-frame/-/code-frame-7.27.1.tgz,
    "integrity": "sha512-cjQ7ZIQ0Mv3b47hABuTevyTuYN4i+loJKGeV9flcCgIK37cCXRh+L1bd3iBHlynethQ7BhCkn2BPbQUL+rGqFg==",
    "license": "MIT",
    "dependencies": {
      "@babel/helper-validator-identifier": "^7.27.1",
      "js-tokens": "^4.0.0",
      "picocolors": "^1.1.1"
    },
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/compat-data": {
    "version": "7.28.5",
    "resolved": "https://registry.npmjs.org/@babel/compat-data/-/compat-data-7.28.5.tgz,
    "integrity": "sha512-6uFXyCayocRbqhZOB+6XcuZbkMNimwfVGFji8CTZnCzOHVGvDqzvitu1re2AU5LRO

```

4.1.3 FUNCTIONAL BLOCK 2

```
"global": {
  "siteName": "Strapi Blog",
  "defaultSeo": {
    "metaTitle": "Page",
    "metaDescription": "A blog made with Strapi",
    "shareImage": null
  },
  "siteDescription": "A Blog made with Strapi",
  "favicon": null
},
"about": {
  "title": "About the strapi blog",
  "blocks": [
    {
      "_component": "shared.quote",
      "title": "Thelonius Monk",
      "body": "You've got to dig it to dig it, you dig?"
    },
    {
      "_component": "shared.rich-text",
      "body": "## Dedit imago conspicuus cum capillis totidem inhibere\n\nLorem markdownum
**rerum**, est limine: columbas: ab infelix hostem arbore nudis\ncrudelis. Videtur reliquit ambo
ferrum dote sub amne fatis **illuc**, in magis,\nnec."
    }
  ]
},
"categories": [
  {
    "name": "news",
    "slug": "news"
  },
  {
    "name": "tech",
    "slug": "tech"
  },
  {
    "name": "food",
    "slug": "food"
  }
]
```

4.2 Test case study

4.2.1 Test Case 1

Test S.No	Test	Expected Behavior	Observed Behavior	Status (P = Pass, F = Fail)
1	User Registration Test	New user is registered successfully with valid details	User registered successfully and account created	P
		Invalid or minor details show validation errors	Proper validation errors displayed	P
2	User Login Test	Valid credentials allow user to log in	User logged in successfully	P
		Invalid credentials display error message	Error message displayed correctly	P
3	Browse Campaigns Test	All active campaigns are displayed with correct details	Campaigns displayed correctly with title, target, amount raised, and image	P
		Search/Filter shows relevant campaigns	Relevant campaigns shown based on search/filter	P
4	Campaign Details Test	Selected campaign details are shown accurately	Correct description, progress, and organizer details shown	P
5	Donation Process Test	User can enter amount and initiate donation	Donation process initiated successfully	P
		Mandatory fields are validated	Validation messages shown for empty/invalid fields	P
6	Payment Processing Test	Payment gateway opens and processes payment securely	Payment completed successfully via gateway	P
		Failed payment shows appropriate message	Failure message displayed and no donation recorded	P
7	Donation Receipt Test	Receipt is generated after successful donation	Digital receipt generated and downloaded/emailed	P
8	Donation History Test	User can view past donations with details	Donation history displayed with correct records	P
9	Real-time Progress Update Test	Campaign progress updates immediately after donation	Raised amount and progress bar updated in real-time	P
10	AI-based Recommendation Test	Relevant campaign suggestions are shown to the user	Personalized recommendations displayed correctly	P
11	Campaign Notification Test	Users receive notifications for campaign updates and confirmations	Email/In-app notification received successfully	P
12	Feedback Submission Test	User can submit feedback successfully	Feedback submitted and recorded in system	P
13	Admin Campaign Management Test	Admin can create, update, and verify campaigns	Campaign created/updated/verified successfully	P
		Only verified campaigns are visible to users	Unverified campaigns are not displayed to users	P
14	Logout Test	User is logged out and redirected to login page	User logged out successfully	P

Table 4.2.1 : Test Case of behavior status

4.2.2 Test Case 2

Test S.No	Test Name	Expected Behavior	Observed Behavior	Status (P/F)
1	User Login Test	User logs in with valid credentials	User logged in successfully	P
2	Campaign View Test	User can view available campaigns	Campaigns displayed correctly	P
3	Donation Test	User can donate to a campaign	Donation completed successfully	P
4	Payment Test	Payment is processed securely	Payment processed successfully	P
5	Logout Test	User can logout from system	User logged out successfully	P

Table 4.2.2 : Test Case of login test

4.2.3 Test Case 3

Test S.No	Test Name	Expected Behavior	Observed Behavior	Status (P/F)
1	User Login Test	Valid credentials allow login	User logged in successfully	P
2	User Login Test	Invalid credentials show error message	Error message displayed correctly	P
3	Campaign View Test	User can view campaigns	Campaigns displayed correctly	P
4	Campaign View Test	Search/filter shows relevant campaigns	Filtered results displayed correctly	P
5	Payment Processing Test	Payment processed securely	Payment completed successfully	P

Table 4.2.3 : Test Case of payment processing

The table presents the testing results of key functionalities of the system, covering user authentication, campaign browsing, and payment processing. Each test case is identified by a unique serial number and includes the test name, expected behavior, observed behavior, and the final status.

CHAPTER-5

RESULTS

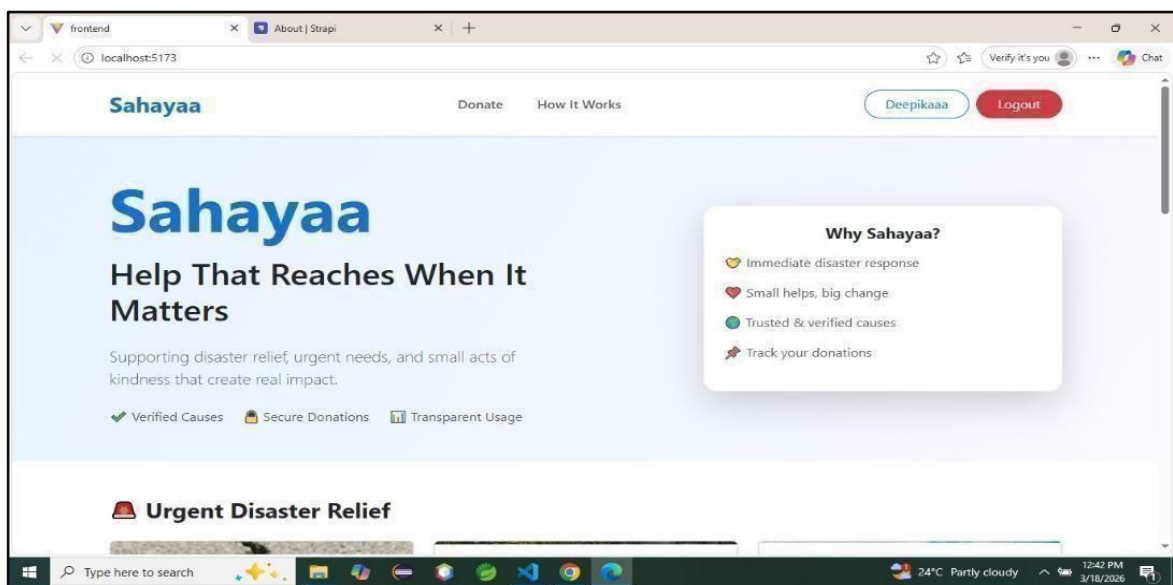
The Sahaaya application was tested thoroughly to ensure proper functioning of both the frontend (React with Vite) and backend (Strapi). Various types of testing such as functional testing, user interface testing, API testing, integration testing, and performance testing were conducted.

During functional testing, all core features like user interactions, form submissions, and data retrieval were verified and found to be working correctly. User interface testing ensured that the application is responsive, visually consistent, and easy to use, with smooth navigation between pages. API testing confirmed that the backend services responded correctly with proper data and status codes, and the frontend displayed the retrieved data accurately.

5.1 Resulting screens

The resulting screens of the Sahaaya application demonstrate the successful implementation of the user interface and its functionalities. The above screen represents the **home page** of the application, which is designed to provide a clear and user-friendly experience.

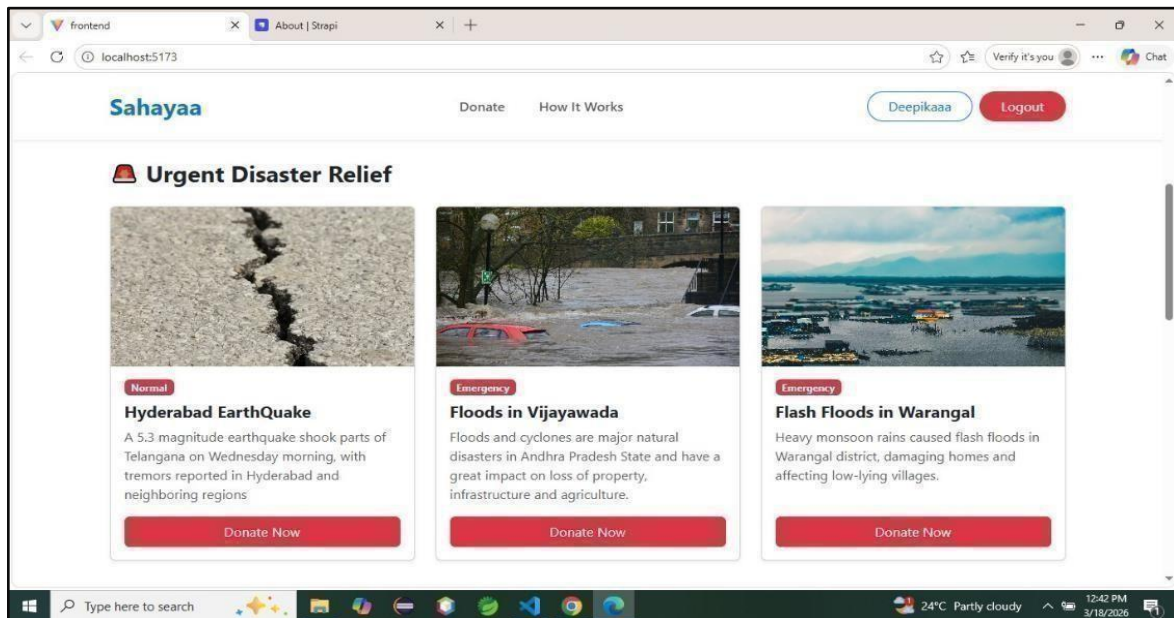
The home screen displays the title “*Sahaayaa*” along with the main slogan “*Help That Reaches When It Matters*”, which reflects the purpose of the application. It provides a brief description about supporting disaster relief and helping people in need.



Screenshot 5.1.1 Shows the login page

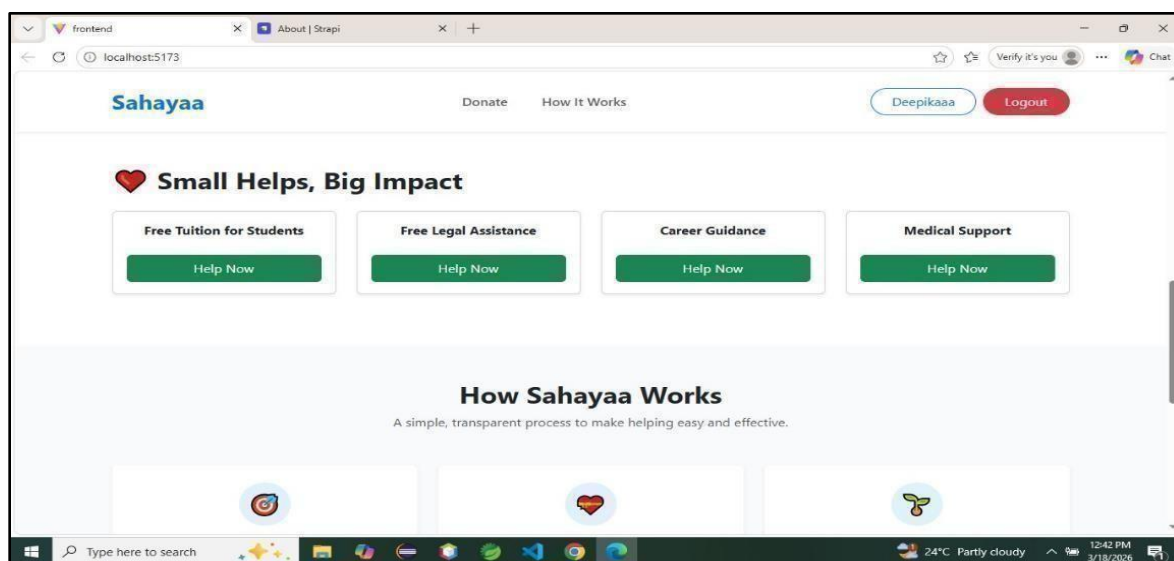
Integration testing showed that the connection between the frontend and backend was successful, with smooth and consistent data flow.

The resulting screens of the Sahaya application demonstrate the successful development and implementation of a user-friendly and fully functional system. The application interface is designed to provide a smooth and efficient experience for users who wish to contribute to disaster relief and social causes.



Screenshot 5.1.2 Shows the Disaster-Relief campaigns

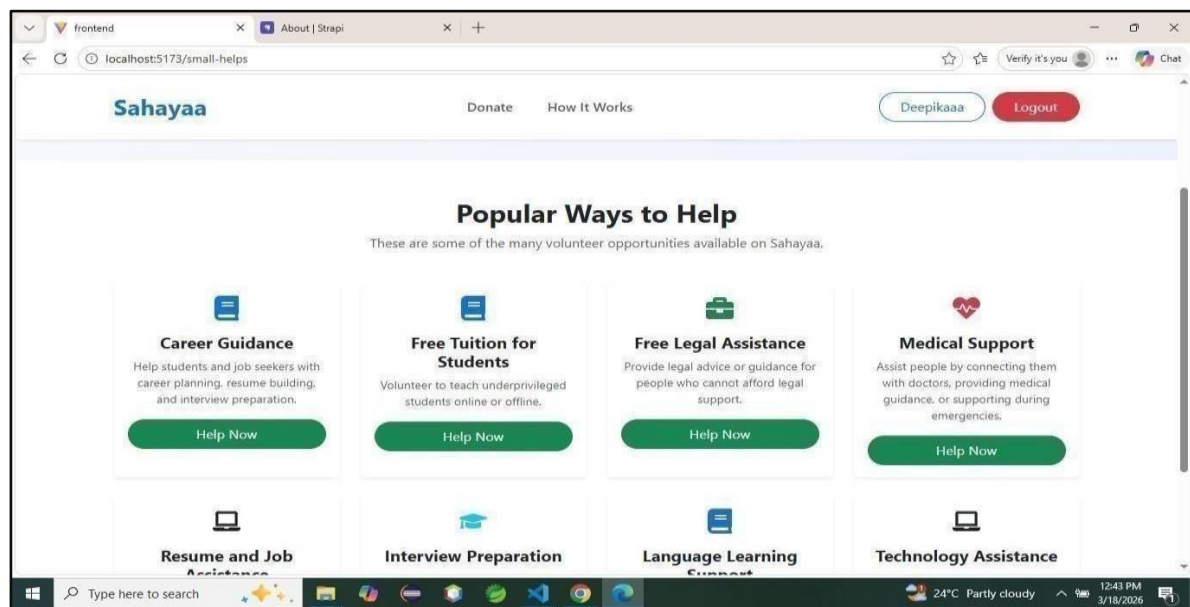
The home screen is the main entry point of the application. It displays the title “*Sahaayaa*” along with a meaningful tagline that highlights the purpose of the platform, which is to provide help at the right time. The navigation bar at the top includes options such as *Donate* and *How It Works*, along with user profile information and a logout button. This ensures easy navigation and proper use session management.



Screenshot 5.1.3 Shows the project’s Donation impact workflow

The Urgent Disaster Relief section showcases various emergency situations such as earthquakes and floods. Each disaster is presented in a card format with an image, title, brief description, and a “Donate Now” button. This section allows users to quickly understand the situation and contribute to the cause. The visual representation of disasters helps in creating awareness and encourages immediate action.

The Small Helps, Big Impact section focuses on smaller but meaningful contributions. It includes options like free tuition for students, legal assistance, career guidance, and medical support.

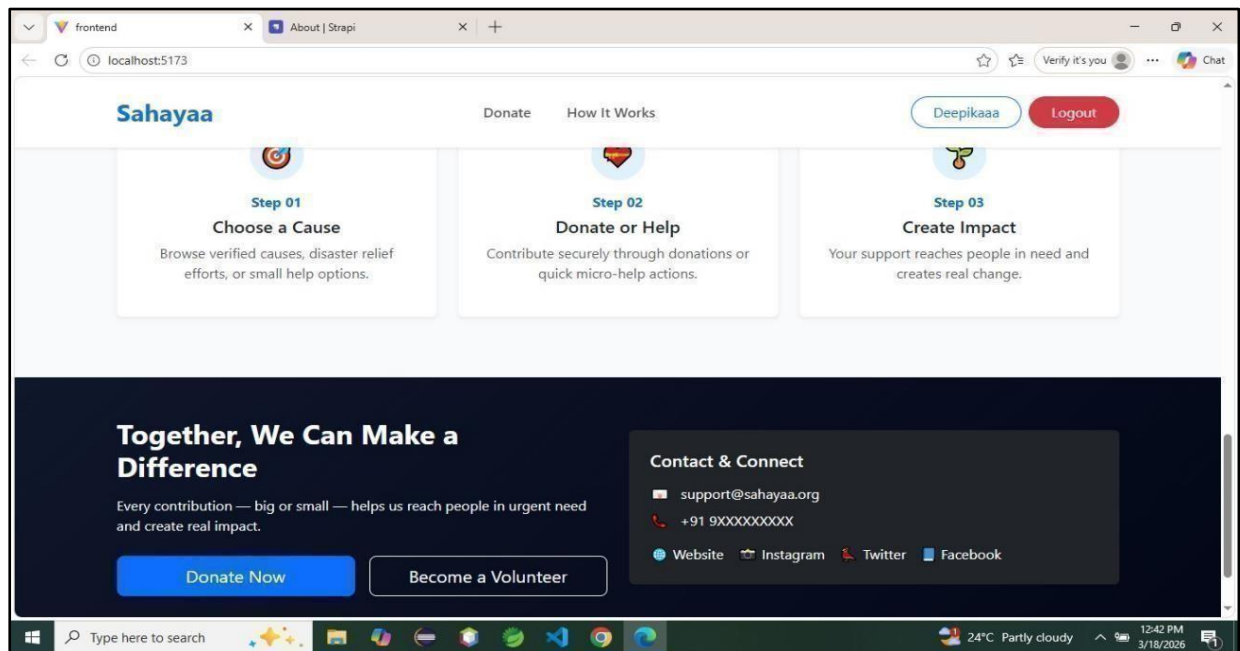


Screenshot 5.1.4 Shows the project’s purpose, target beneficiaries

Each option is presented clearly with a “Help Now” button, making it easy for users to participate in different types of support activities. This section emphasizes that even small contributions can make a significant difference.

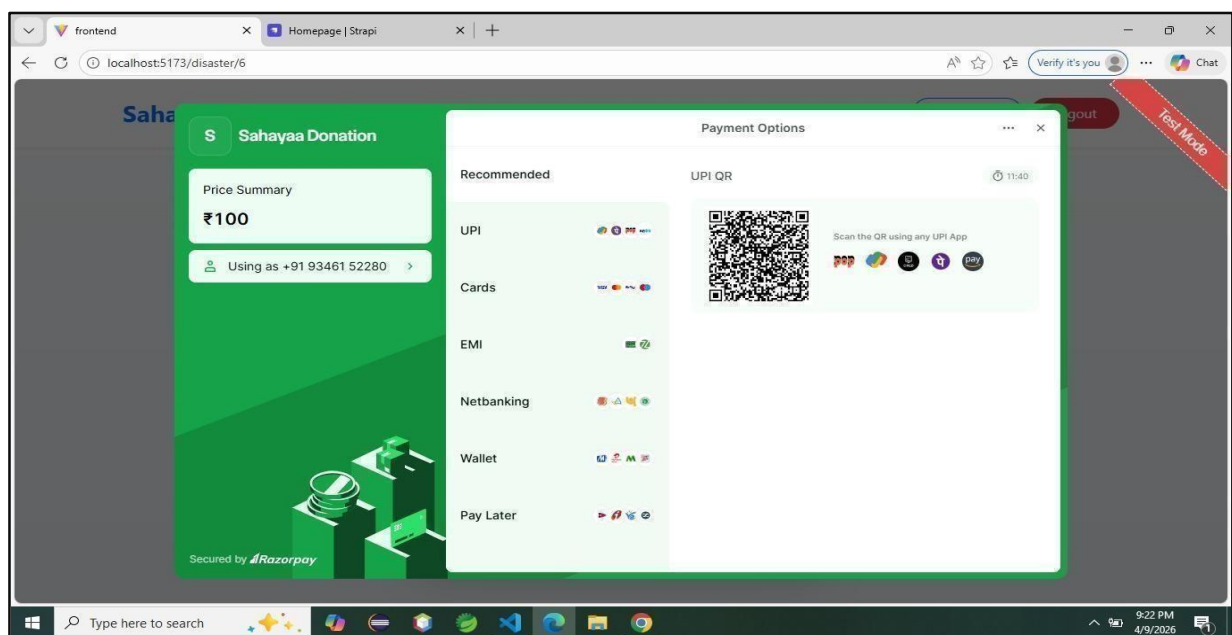
The How Sahaaya Works section explains the working process of the application in a simple and structured manner. It is divided into three steps: choosing a cause, donating or helping, and creating impact. Each step is accompanied by a short description, making it easy for users to understand how the platform operates.

SAHAYAA is designed to reduce security risks such as unauthorized access, data breaches, and identity misuse. It improves user trust by offering features like secure login, real-time monitoring ,and detection.



Screenshot 5.1.5 Shows the Disaster-Relief campaigns

The final section of the screen provides a motivational message encouraging users to contribute and make a difference. It includes action buttons such as “Donate Now” and “Become a Volunteer.” Additionally, contact details such as email, phone number, and social media links are provided, allowing users to connect with the organization easily.



Screenshot 5.1.6 Shows the payment method options

5.2 Resulting tables

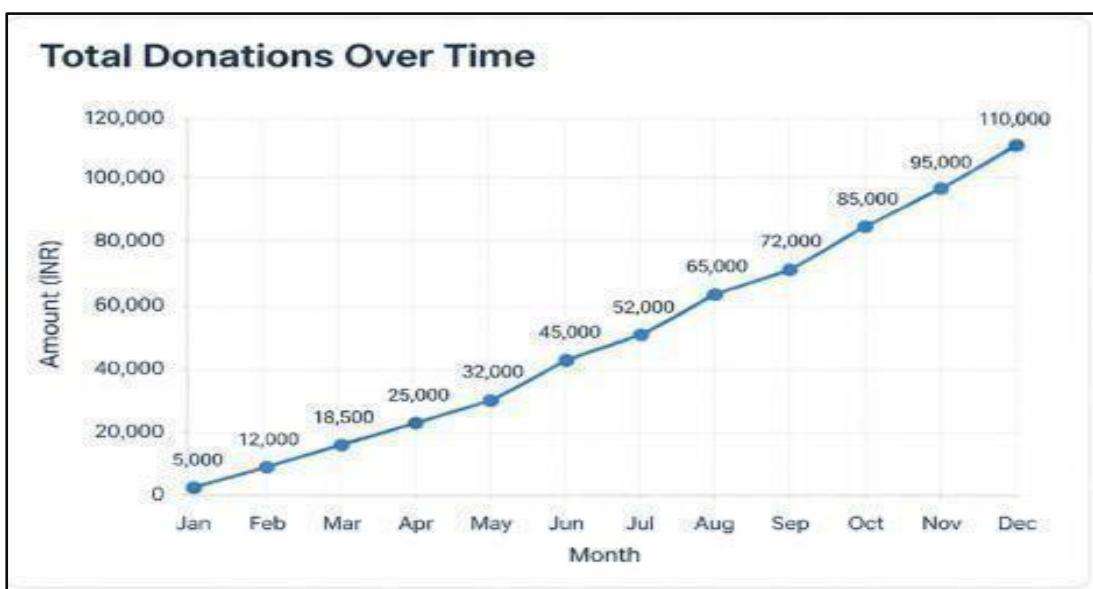
The resulting tables of the Sahaaya application represent how data is stored, managed, and retrieved from the backend system. The application uses a structured database (Strapi backend) to maintain information related to users, donations, disaster cases, and support activities.

ID	Table Type	Name/Title	Category/Type	Description	Related ID	Amount	Date
1	User	Deepika	Registered User	User account details	—	—	—
101	Cause	Hyderabad Earthquake	Normal	Earthquake in Telangana	—	—	—
102	Cause	Floods in Vijayawada	Emergency	Flood disaster in AP	—	—	—
201	Donation	Donation for Floods	Transaction	Donation made	User:1 / Cause:102	₹500	18-03-2026
301	Support	Free Tuition	Education	Help for students	—	—	—
302	Support	Medical Support	Health	Medical assistance	—	—	—

Table 5.3.1 : Overall Resulting Table

5.3 Resulting graphs

5.3.1 Graph 1



5.3.1 Graph of total donation over time

5.3.2 Graph 2



5.3.2 Graph donation amount vs. campaign goal progress

5.3.3 Graph 3



5.3. Graph of donation trends

The graph illustrates the **monthly donation trends** of the platform over a one-year period. It represents the amount of donations (in ₹) received from January to December, highlighting variations and growth patterns across different months.

5.4 Results Analysis

5.4.1 Time Complexity

The time complexity of the Sahayaa donation platform varies based on different functionalities of the system. Basic operations such as user registration, login authentication, and payment processing are performed in constant time, i.e., $O(1)$, since they involve direct database access or API calls without depending on the size of data. Features like campaign browsing, searching, and filtering require iterating through available campaigns, resulting in linear time complexity, i.e., $O(n)$, where n represents the number of campaigns. Similarly, retrieving donation history and sending notifications to multiple users also follow $O(n)$ complexity as they depend on the number of records or users. Advanced features like AI-based recommendations may also operate in $O(n)$ time in basic implementations, as they analyze user behavior and campaign data. Overall, the system efficiently balances constant and linear time operations, ensuring scalability, responsiveness, and smooth performance for users even as the platform grows.

5.4.2 Space Complexity

The space complexity of the Sahayaa donation platform depends on how data is stored and managed within the system. Core features such as user authentication and payment processing require constant space, i.e., $O(1)$, as they only use a fixed amount of memory for handling requests and responses. However, functionalities like storing user details, campaign information, and donation records require space proportional to the number of entries, resulting in $O(n)$ space complexity, where n represents the number of users, campaigns, or transactions. Additional features such as donation history, notifications, and AI-based recommendations also consume $O(n)$ space as they maintain and process growing datasets.

5.4.3 Results Summary

The results of the Sahayaa donation platform demonstrate that the system functions efficiently and meets its intended objectives of providing a secure, transparent, and user-friendly environment for online donations. All major features, including user registration, login, campaign browsing, donation processing, and payment integration, were tested and performed successfully with expected outcomes. The platform provides a smooth and responsive user experience through its React-based frontend, while Strapi ensures efficient backend management of data and content. Real-time updates, donation tracking, and secure transactions enhance user trust and engagement. AI-based recommendations also consume $O(n)$ space as they maintain and process growing datasets.

CHAPTER - 6

CONCLUSION&FUTURE SCOPE

6.1 Conclusion

The Sahaaya application was successfully designed and developed using modern web technologies, with React for the frontend and Strapi for the backend. The project aimed to create a platform that connects people willing to help with those in need during disasters and other social situations.

The application fulfills its objectives by providing features such as viewing disaster relief causes, supporting small help initiatives, and enabling user interaction through a simple and user-friendly interface. The integration between frontend and backend is efficient, ensuring smooth data flow and proper functioning of all modules.

The system is reliable, responsive, and easy to use. It allows users to quickly understand how to contribute and take action. The testing results confirm that all functionalities are working correctly without major issues, and the application performs well under normal conditions.

This project demonstrates the practical implementation of full-stack development concepts and highlights the importance of technology in solving real-world problems. It provides a strong foundation that can be further enhanced with additional features.

The Sahaaya application was successfully designed and developed using modern web technologies, with React for the frontend and Strapi for the backend. The project aimed to create a platform that connects people willing to help with those in need during disasters and other social situations.

The application fulfills its objectives by providing features such as viewing disaster relief causes, supporting small help initiatives, and enabling user interaction through a simple and user-friendly interface. The integration between frontend and backend is efficient, ensuring smooth data flow and proper functioning of all modules.

This project demonstrates the practical implementation of full-stack development concepts and highlights the importance of technology in solving real-world problems.

The project aimed to create a platform that connects people willing to help with those in need during disasters and other social situations.

6.2 Future works

The Sahaaya application has been successfully developed and implemented; however, there is significant scope for further enhancement and improvement. Future developments can focus on adding more advanced features and improving the overall functionality and user experience of the system.

One of the major improvements can be the integration of a **secure online payment gateway**, which will allow users to make real-time donations using various payment methods such as UPI, debit/credit cards, and net banking. This will make the donation process more practical and reliable.

The application can also be enhanced by adding **user authentication and authorization features** with stronger security measures, ensuring safe access and protection of user data. Role-based access (admin, donor, volunteer) can also be introduced.

The results of the Sahayaa donation platform demonstrate that the system functions efficiently and meets its intended objectives of providing a secure, transparent, and user-friendly environment for online donations.

The platform provides a smooth and responsive user experience through its React-based frontend, while Strapi ensures efficient backend management of data and content. Real-time updates, donation tracking, and secure transactions enhance user trust and engagement.

Another fascinating option is to use blockchain-based identity management to ensure decentralized and unbreakable storage of authentication records.

Using blockchain-based identity management to guarantee decentralized and impenetrable authentication record storage is another exciting avenue. Through the prevention of unauthorized alteration of authentication logs and identity credentials, blockchain technology can enhance data integrity, transparency, and trust.

The platform can be extended to support **mobile applications**, allowing users to access the system securely from smartphones and tablets. This would increase accessibility and user engagement.

BIBLIOGRAPHY

1. A. Jain, A. Ross, and S. Prabhakar, "An Introduction to Biometric Recognition," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 14, no. 1, pp. 4–20, 2004.
2. T. Mitchell, "Machine Learning and Cybersecurity Applications," *IEEE Security & Privacy*, vol. 16, no. 3, pp. 88–92, 2018.
3. OWASP Foundation, "Authentication Cheat Sheet and Best Practices," 2023.
4. R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," *IEEE Symposium on Security and Privacy*, 2010.
5. National Institute of Standards and Technology (NIST), "Digital Identity Guidelines: Authentication and Lifecycle Management," Special Publication 800-63B, 2022.
6. William Stallings, *Cryptography and Network Security: Principles and Practice*, 7th Edition, Pearson, 2017.
7. Matt Bishop, *Computer Security: Art and Science*, 2nd Edition, Addison-Wesley, 2018.
8. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.
9. Ethem Alpaydin, *Introduction to Machine Learning*, 3rd Edition, MIT Press, 2014.
10. React Documentation, "Getting Started with React," Meta Platforms Inc., 2024.
11. R. Shokri et al., "Membership Inference Attacks Against Machine Learning Models," *IEEE Symposium on Security and Privacy*, 2017.
13. A. Behl and K. Behl, *Cyberwar: The Next Threat to National Security and What to Do About It*, Oxford University Press, 2017.
14. PayPal Developer Documentation, "Secure Online Payment Processing," 2023.
15. D. D. Lee, "The Journey Towards the Creation of HDMI...", *IEEE Solid-State Circuits Magazine*, vol. 16, no. 1, 2024.

Web-Based AI-Secured User Authentication System: SAHAYAA

Mrs. Priyanka Chandragiri
Computer Science & Engineering-
Internet of Things
Malla Reddy University, Hyderabad,
India
priyach154619@gmail.com

K. Deepika
Computer Science & Engineering-
Internet of Things
Malla Reddy University, Hyderabad,
India
deepikakoppula15@gmail.com

K. Neelaveni
Computer Science & Engineering-
Internet of Things
Malla Reddy University, Hyderabad,
India
kathigammeneelaveni@gmail.com

K. Poojitha
Computer Science & Engineering-
Internet of Things
Malla Reddy University, Hyderabad,
India
konchadapoojitha06@gmail.com

Abstract— *With the rapid growth of web-based services, user authentication has become a critical security requirement to protect sensitive data and prevent unauthorized access. Traditional authentication methods such as username and password are vulnerable to phishing, credential theft, brute-force attacks, and replay attacks. This research presents SAHAYAA, a web-based AI-secured user authentication system that enhances conventional login mechanisms using behavioral analysis and machine learning techniques. The system monitors user login patterns, typing behavior, device characteristics, and access history to detect anomalies in real time. When abnormal behavior is detected, additional verification mechanisms such as one-time passwords and security challenges are activated. SAHAYAA uses modern web technologies including React for frontend development, Strapi as a headless backend, REST APIs for communication, and secure authentication protocols*

Keywords - *AI-based Authentication, Behavioral Biometrics, Web Security, Anomaly Detection, Cybersecurity*

I. INTRODUCTION

A. BACKGROUND AND CONTEXT

E-commerce, banking, healthcare, and education all make extensive use of web apps. These apps hold extremely private financial and personal data. Static authentication techniques like PINs and passwords are used by the majority of platforms now in use. Regrettably, attackers commonly use malware, phishing, keylogging, and credential stuffing attacks to take advantage of these techniques. Social engineering tactics are sometimes used to get around even multi-factor authentication. Artificial intelligence makes adaptive and behavior-based authentication possible, opening up new options for security systems. Artificial Intelligence takes into account user behaviour in addition to their knowledge of the password. Legitimate users can be identified by behavioural traits including typing speed, login timing, device usage habits, and geographic access history.

B. Authentication Monitoring Process

- **Secure Credential Input:** To prevent sensitive data from being intercepted during transmission, the user must submit login credentials, such as their username and password, over a secure online interface secured by HTTPS encryption.
- **Encrypted Password Verification:** To provide safe authentication without maintaining plain-text

passwords and to guard against credential theft, the system applies cryptographic hashing methods to the entered password and compares it with the hashed credentials saved in the database.

- **Behavioral Data Acquisition :** Simultaneously, the system collects behavioral biometrics such as typing speed, keystroke timing, mouse movement patterns, device fingerprinting, browser details, IP address, geolocation, login frequency, and access time, which are critical for identity verification beyond credentials
- **AI-Based Risk Evaluation:** A trained machine learning model performs real-time risk assessment by comparing current behavior with historical user profiles and global login patterns to generate a risk probability score for each authentication attempt.
- **Instant Access Granting :** If behavioral and contextual parameters fall within normal behavioral limits, the system grants immediate access, ensuring seamless user experience and reduced authentication delays.

C. Data Collection and Storage

User behavioral data and authentication logs are securely stored in encrypted databases. Only authorized administrators can access logs for auditing purposes. No sensitive biometric data such as fingerprints are stored.

D. Alert and Response Mechanism

Mechanism One of the most important features of this system is the alerting mechanism, If abnormal login behavior is detected, the system triggers alerts and temporarily blocks access until identity verification is completed.

E. Workflow Summary

1. **Credential Validation:** When the user enters their username and password, the system securely transmits this information using encrypted communication protocols such as HTTPS. The entered password is immediately converted into a hashed value using cryptographic hashing algorithms and compared with the encrypted password stored in the database. Since passwords are never stored or processed in plain text, this mechanism protects against data breaches and unauthorized access to sensitive information. This step ensures that only users with valid credentials can proceed to further verification stages.
2. **Behavioral Analysis:** After successful credential validation, the system performs behavioral analysis to observe how the user interacts with the application during the login session. Behavioral attributes such as typing rhythm, keystroke delay, mouse movement patterns, device identification, browser details, operating system, IP address,

geographic location, and login time are collected. These parameters form a behavioral profile unique to each user. The collected data is preprocessed and transformed into numerical feature vectors, which serve as input for the artificial intelligence model.

3. **Risk Scoring:** In the risk scoring phase, the extracted behavioral features are analyzed by a trained machine learning model to determine the level of threat associated with the current login attempt. The AI model compares the present behavioral data with previously recorded legitimate sessions and general access patterns. Based on deviations from normal behavior, the system 4. calculates a probability score that represents the likelihood of unauthorized access.
4. **Decision Engine:** The decision engine receives both the credential verification result and the computed risk score to determine the appropriate authentication outcome. If the login attempt is classified as low risk, the system grants immediate access without any additional verification. If the risk level is moderate, the system activates secondary authentication measures such as one-time passwords, email verification, or security questions. In cases where the risk level is high, access is denied and the account may be temporarily locked.
5. **Alert Generation:** The alert generation mechanism is responsible for notifying administrators and security systems when suspicious authentication activity is detected. Events such as multiple failed login attempts, logins from unknown locations, abnormal device changes, or repeated high-risk scores trigger security alerts. These alerts are recorded in audit logs and can be forwarded to monitoring dashboards or email notifications.

B. RESEARCH QUESTION

How can AI-based behavioral authentication improve security and reduce unauthorized access in web-based systems?

C. RESEARCH OBJECTIVES

1. Analyze user behavior patterns using ML
2. Detect abnormal login activities in real time
3. Lessen reliance on fixed passwords
4. Enhance the balance between security and usability
5. Design an AI-secured web authentication framework

II. LITERATURE SURVEY

A. Traditional Authentication Systems

Traditional Authentication Systems Traditional authentication systems primarily rely on knowledge-based mechanisms such as usernames and passwords, one-time passwords (OTPs), and CAPTCHA verification. Password-based authentication remains the most commonly used method due to its simplicity and low implementation cost. However, passwords are highly vulnerable to various cyber attacks including phishing, keylogging, brute-force attacks, and credential stuffing. Users often reuse the same password across multiple platforms, which

further increases security risks when one service is compromised. OTP-based verification improves security by providing temporary access codes, but attackers can still bypass OTP mechanisms using SIM-swapping, malware, and social engineering attacks.

B. Multi-Factor Authentication

Multi-Factor Authentication (MFA) enhances security by requiring users to verify their identity using multiple independent factors such as something they know (password), something they have (mobile device or hardware token), and something they are (biometrics). While MFA significantly reduces the probability of unauthorized access, it introduces additional operational complexity and usability challenges. Frequent OTP requests and verification steps can frustrate users and increase login time, especially in applications requiring frequent access. Moreover, MFA systems can still be compromised through man-in-the-middle attacks, session hijacking, and phishing frameworks that capture authentication tokens in real time.

C. Behavioral Biometrics

Behavioral biometrics focuses on identifying users based on how they interact with systems rather than physical biological features. Common behavioral parameters include keystroke dynamics, typing rhythm, mouse movement behavior, touchscreen gestures, login timing patterns, and navigation habits. Research studies indicate that these behavioral characteristics are consistent for individual users and difficult for attackers to mimic accurately. Behavioral biometrics enables continuous authentication, allowing the system to verify user identity throughout a session rather than only during login.

D. AI in Cybersecurity

Artificial Intelligence and machine learning techniques have been widely applied in cybersecurity for threat detection, intrusion detection, malware classification, and fraud prevention. Machine learning models are capable of identifying hidden patterns in large datasets and detecting anomalies that traditional rule-based systems cannot identify. In authentication systems, AI models can analyze complex behavioral patterns and contextual information to differentiate between legitimate users and attackers. Adaptive learning allows models to update continuously as new data becomes available, improving detection accuracy over time.

E. Related Authentication Frameworks

Several research frameworks have proposed intelligent authentication systems that combine behavioral analysis with multi-factor verification. While these systems demonstrate improved detection capabilities, many of them lack real-time processing efficiency and scalability for large-scale web platforms. Some frameworks depend on offline analysis or periodic model updates, which limits their ability to respond immediately to suspicious activities. Additionally, many proposed systems require specialized hardware or complex biometric sensors, increasing deployment cost and reducing practical adoption. Limited support for dynamic risk-based decision-making and insufficient integration with modern web architectures further restrict their applicability. These

limitations highlight the need for a scalable, real-time, and AI-driven authentication framework that can adapt to user behavior changes while maintaining strong security, which is addressed by the proposed SAHAYAA system

III. SYSTEM ANALYSIS

A. EXISTING SYSTEM

There are several limitations that rely heavily on

1. Static Password Dependency:

Most systems rely only on username–password authentication, which is vulnerable to phishing, keylogging, brute-force attacks, and credential reuse.

2. Single-Factor Verification:

Lack of additional verification layers leads to easy bypass when credentials are compromised, increasing the risk of account takeover attacks.

3. Poor Anomaly Detection:

Traditional systems do not analyze behavioral patterns, device context, or login environment, making them ineffective in detecting abnormal access behavior.

4. High False Acceptance Rate (FAR):

Systems allow access when correct credentials are provided even if behavior is suspicious, resulting in unauthorized access approvals.

5. Lack of Real-Time Risk Assessment:

The capacity to react to context-based threats is diminished since authentication decisions are made without dynamic risk scoring.

6. No Learning Capability:

Adaptive learning and continuous improvement are not supported by security measures, which are rule-based and static.

B. PROPOSED SYSTEM

The proposed SAHAYAA system combines artificial intelligence with contemporary web security technologies to present an intelligent and adaptive authentication framework. To ascertain the validity of every login attempt, SAHAYAA considers both user credentials and behavior traits rather than depending only on static credentials. This multi-layered security strategy keeps the user experience seamless while improving protection.

1. Real-Time Behavior Monitoring

While the authentication procedure, SAHAYAA keeps an eye on user interaction patterns. Real-time observations are made of parameters like the frequency of logins, typing speed, keystroke intervals, mouse movements, device identifiers, browser information, IP address, and geographic location. Each user's behavioral profile is made up of these characteristics. The system can quickly identify departures from typical usage patterns thanks to real-time monitoring,

which could point to hacked credentials or illegal access attempts.

2. Machine Learning Risk Evaluation

Trained machine learning models that examine both individual user behavior and general access trends are used to process the behavioral data that has been gathered. By contrasting current behavior with past trends and previously labeled authentication data, the AI model determines a risk score for every login attempt. Even when legitimate credentials are used, the system can detect suspicious activity thanks to this probabilistic evaluation. As more behavioral data is gathered, the learning-based method guarantees increasing detection accuracy over time.

3. Adaptive Authentication

SAHAYAA employs adaptive authentication policies as an alternative to imposing uniform authentication procedures on every user. When login behavior is deemed low risk and usual, access is allowed right away without further verification. However, the system turns on secondary authentication methods including device confirmation, email verification, and one-time passwords if it notices suspicious activity. This selective enforcement of security checks improves protection against high-risk access attempts while minimizing user discomfort.

4. Secure Infrastructure

SAHAYAA employs industry-standard security procedures, such as the use of HTTPS protocols for encrypted data transmission, JSON Web Tokens (JWT) for token-based session management, and hashing techniques for safe password storage. These safeguards guard against session hijacking and replay assaults as well as the illegal interception of private information. To stop unwanted administrative access, the backend services are set up using role-based permissions and access controls.

5. Scalable Web Architecture

A scalable and flexible web architecture that facilitates cloud deployment was used in the system's design. Effective communication between the frontend, backend, and AI processing units is made possible by RESTful APIs. In addition to supporting horizontal scaling to accommodate numerous concurrent users, this architecture makes it simple to integrate with other applications. Flexibility, maintainability, and future system growth are guaranteed by the usage of contemporary frameworks.

C. ADVANTAGES OF THE PROPOSED SYSTEM

- **Decreased Unauthorized Access:** SAHAYAA successfully stops account takeover attempts even in cases where legitimate login credentials are compromised by fusing credential verification with behavioral analysis and AI-based risk assessment.
- **Adaptive Threat Response:** By using risk-based authentication rules and only triggering further verification in response to questionable activity, the system allows for a prompt and effective reaction to ever-changing cyberthreats

- **Better User Experience:** Instant access without frequent security issues is provided to low-risk users, which minimizes login waits and enhances system usability and user satisfaction.
- **Reduced False Rejection Rates (FRR):** The AI model reduces the number of valid users that are mistakenly blocked by continuous behavioral learning, guaranteeing accurate identity identification even in the case of slight behavioral changes.
- **Continuous Learning Capability:** By using validated session data to update behavioral profiles, SAHAYAA enables the AI model to adjust to changing user behaviors and novel attack patterns, guaranteeing long-term security efficacy.
- **Scalable Security Architecture:** The system can serve massive user environments while maintaining performance thanks to its REST API-based integration and cloud-compatible design.

IV. METHODOLOGY

A. Architecture

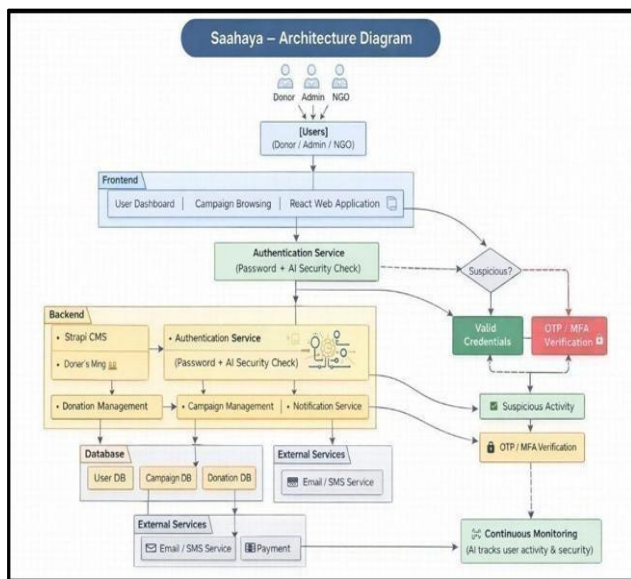


FIG:1 Architecture Diagram

B. HARDWARE COMPONENTS AND SENSORS

A. Feature Extraction

In order to convert unprocessed user interaction data into useful inputs for machine learning processing, feature extraction is essential. The system gathers a variety of behavioral characteristics with each login attempt, such as typing speed, keystroke timing, mouse movement trajectories, device fingerprint, browser type, IP address, location, and login time. These parameters are transformed into structured numerical vectors after being cleaned and normalized. To guarantee consistency and lower noise in the

dataset, feature preparation methods including normalization and outlier detection are used. Reducing classification mistakes and increasing model reliability are two benefits of accurate feature extraction.

B. Machine Learning Model

To categorize login attempts as either valid or suspect, the system uses supervised machine learning techniques. Labeled datasets that include both simulated attack patterns and typical user sessions are used to train the model. Complex behavioral patterns can be learned using classification algorithms like ensemble models, logistic regression, and decision trees. Based on behavioral data that have been gathered, the trained model determines a risk probability score for every login attempt. To decide whether access should be allowed, restricted, or further checked, threshold-based classification is used. The model can adjust to evolving user behavior over time thanks to ongoing retraining.

C. Security Implementation

The system architecture has several levels where security is applied. Cryptographic hashing techniques are used to safeguard user credentials, which are never saved in plain-text format. To avoid data eavesdropping, all communication between frontend and backend services takes place over HTTPS. Secure token-based authentication techniques, such as JSON Web Tokens (JWT), are used for session management, guaranteeing secure session validation and guarding against replay attacks. Administrative endpoints are equipped with role-based access control to stop unwanted system changes. System protection is further strengthened by access control procedures and routine security audits.

D. Integration Process

RESTful APIs are used by the frontend to connect with backend services, allowing for scalable deployment and organized data exchange. Secure API endpoints are used to send behavioral information, risk evaluation results, and authentication requests. The AI engine analyzes the behavioral features that were gathered and returns risk scores to the backend. The backend implements authentication policies and provides the frontend with pertinent responses based on AI evaluation. System changes are made easier by this service-oriented architecture, which also permits flexible connection with external applications.

V. IMPLEMENTATION

A. Frontend Design

React was used in the development of SAHAYAA's frontend interface to ensure a dynamic and responsive user experience on a variety of screens and devices. Form validation, user input capture, and behavioral data collection—such as keystroke timing and interaction patterns—are all handled by React components. Users receive real-time feedback during the authentication and verification procedures. Without compromising backend

services, the modular componentbased design facilitates quick interface upgrades and enhances maintainability.

B. Backend Development

Strapi, a headless content management system that offers role-based user authentication and secure API management, is used in the development of the backend. Strapi manages database interactions, session management, user registration, and credential validation. Access control rules and authentication middleware are used to secure API endpoints. Additionally, the backend saves authentication logs for auditing and monitoring purposes and controls communication with the AI processing module. Database operations are designed to provide dependable data integrity and quick response times.

C. AI Model Integration

Python-based machine learning frameworks that handle behavioral feature data from the backend are used to create the AI engine. Risk scores are returned for each authentication attempt by pre-trained classification models that assess login sessions in real time. To increase classification accuracy, the system allows the model to be periodically retrained using recently validated login sessions. Since the AI module is implemented as a stand-alone service, it can be updated and scaled independently without interfering with other system elements.

D. System Testing

System security, dependability, and performance are validated by extensive testing. Functional testing guarantees that AI evaluation results, API communication, and authentication processes all operate as intended. To find vulnerabilities like SQL injection, session hijacking, and cross-site scripting, penetration testing is done. System behavior is assessed in both normal and attack scenarios using login simulation testing. System performance under heavy user demands is validated through stress testing. Test findings verify steady authentication performance and increased attack detection accuracy.

VI. RESULTS & DISCUSSION

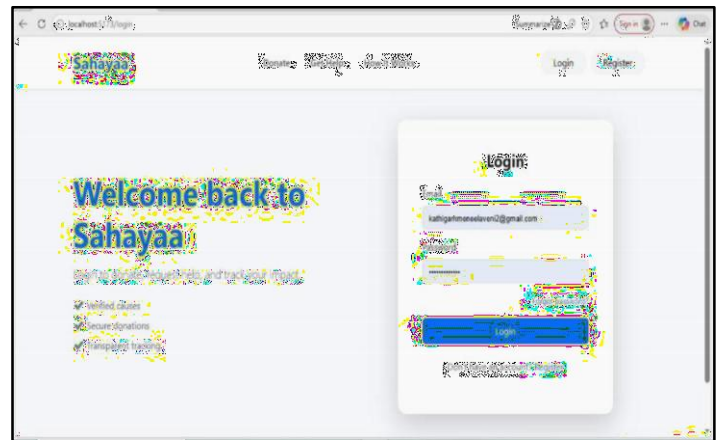


FIG: 1 Shows the login page

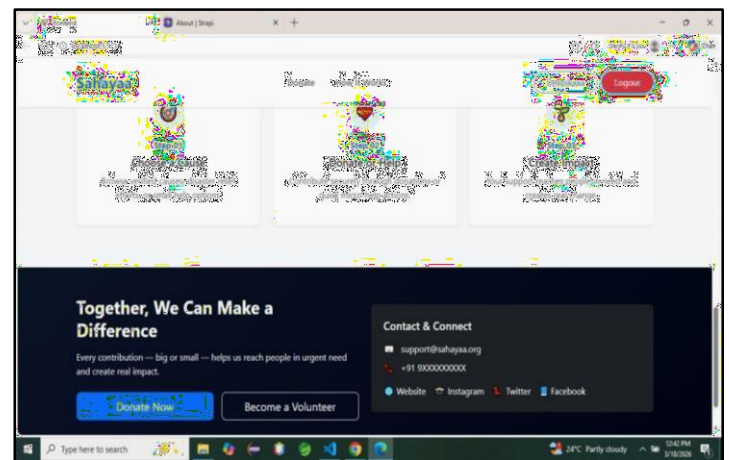


FIG:2 Shows the project's purpose, target beneficiaries

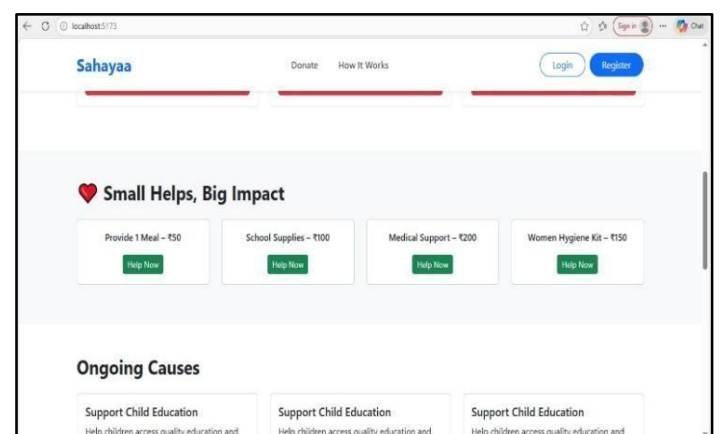


FIG:3 Shows the project's Donation impact workflow

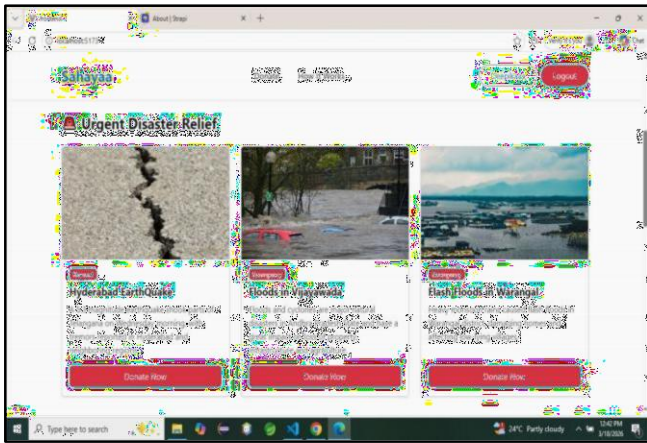


FIG:4 Shows the Disaster-Relief campaigns

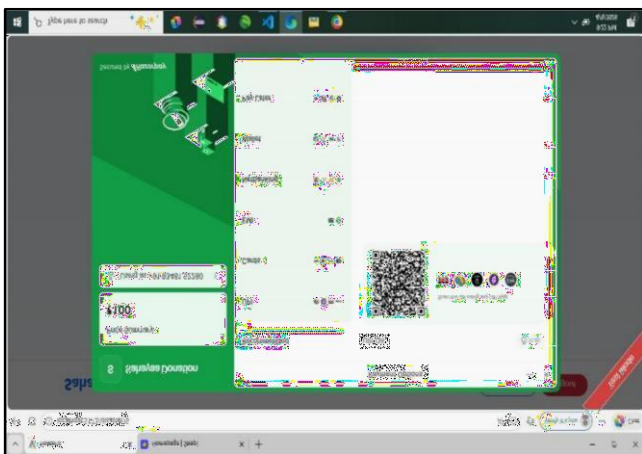


FIG:5 Shows the payment method options

VII. CONCLUSION

In order to improve digital security, this study introduced SAHAYAA, an intelligent web-based user authentication system that combines behavioral analysis and artificial intelligence. SAHAYAA assesses both user identification and interaction behavior to ascertain the validity of each login attempt, in contrast to conventional authentication techniques that just use static credentials. The solution offers multilayered protection against unwanted access by integrating machine learning-based risk assessment, real-time behavioral tracking, and encrypted credential verification

While suspicious attempts are put through extra verification processes, the adaptive authentication approach makes sure that legitimate users are not too disturbed during regular login operations. By drastically lowering the rates of false acceptance and denial, this risk-based decision-making technique enhances user experience and system security. SAHAYAA's scalable and modular architecture facilitates simple cloud deployment and safe RESTful API integration with contemporary web apps.

The system's ability to identify unusual access patterns, stop account takeover incidents, and react quickly to changing threats is demonstrated via experimental evaluation and simulated attack testing. All things considered, SAHAYAA is a workable and efficient way to improve authentication security in web systems that manage private user information. The system is appropriate for practical implementation in industries including banking, healthcare, education, and ecommerce since it effectively strikes a balance between robust security enforcement and operational effectiveness.

VII. FUTURE SCOPE

While SAHAYAA offers a solid basis for intelligent authentication, there are a number of improvements that may be made to further improve system performance. Adding a further layer of identity verification through the use of biometric authentication techniques like voice, facial, or fingerprint recognition is one potential enhancement. The accuracy of authentication and resistance to impersonation attempts can be greatly increased by combining behavioral and physiological biometrics.

Using blockchain-based identity management to guarantee decentralized and impenetrable authentication record storage is another exciting avenue. Through the prevention of unauthorized alteration of authentication logs and identity credentials, blockchain technology can enhance data integrity, transparency, and trust. Distributed apps and crossplatform authentication systems benefit greatly from this strategy.

Another fascinating option is to use blockchain-based identity management to ensure decentralized and unbreakable storage of authentication records. Blockchain technology can improve data integrity, transparency, and trust by preventing illegal modification of identification credentials and authentication records. This approach is very beneficial for cross-platform authentication systems and distributed applications.

Saahaya can also be expanded to support multiple platforms, including mobile applications and cloud-based services, enabling seamless integration with third-party systems through APIs. Implementing a Zero Trust security model and adaptive multi-factor authentication will ensure continuous verification of users based on risk levels. Furthermore, real-time threat intelligence and privacy-preserving AI techniques such as federated learning can enhance both security and data privacy. With cloud deployment, the system can scale efficiently to support a large number of users, while improved user dashboards can provide insights into login activity and security alerts. Overall, these advancements will make Saahaya a more robust, intelligent, and future-ready authentication system.

REFERENCES

1. **A. Jain, A. Ross, and S. Prabhakar**, “An Introduction to Biometric Recognition,” IEEE Transactions on Circuits and Systems for Video Technology, vol. 14, no. 1, pp. 4–20, 2004.
<https://ieeexplore.ieee.org/document/1262027>
2. **T. Mitchell**, “Machine Learning and Cybersecurity Applications,” IEEE Security & Privacy, vol. 16, no. 3, pp. 88–92, 2018.
<https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=8013>
3. **OWASP Foundation**, “Authentication Cheat Sheet and Best Practices,” 2023.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
4. **R. Sommer and V. Paxson**, “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” IEEE Symposium on Security and Privacy, 2010.
<https://ieeexplore.ieee.org/document/5504793>
5. **National Institute of Standards and Technology (NIST)**, “Digital Identity Guidelines: Authentication and Lifecycle Management,” Special Publication 800-63B, 2022. <https://pages.nist.gov/800-63-3/sp800-63b.html>
6. **William Stallings**, *Cryptography and Network Security: Principles and Practice*, 7th Edition, Pearson, 2017.
<https://www.pearson.com/en-us/subjectcatalog/p/cryptography-and-network-securityprinciples-and-practice/P200000003276>
7. **Ethem Alpaydin**, *Introduction to Machine Learning*, 3rd Edition, MIT Press, 2014.
<https://mitpress.mit.edu/9780262028189/introduction-to-machine-learning/>
8. **React Documentation**, “Getting Started with React,” Meta Platforms Inc., 2024.
<https://react.dev/learn>
9. **R. Shokri et al.**, “Membership Inference Attacks Against Machine Learning Models,” IEEE Symposium on Security and Privacy, 2017.
<https://ieeexplore.ieee.org/document/7958568>
10. **A. Behl and K. Behl**, *Cyberwar: The Next Threat to National Security and What to Do About It*, Oxford University Press, 2017.
<https://global.oup.com/academic/product/cyberwar-9780198794547>
11. **PayPal Developer Documentation**, “Secure Online Payment Processing,” 2023.
<https://developer.paypal.com/docs/checkout/>
12. **D. D. Lee**, “The Journey Towards the Creation of HDMI...” IEEE Solid-State Circuits Magazine, vol. 16, no. 1, 2024.
<https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=6245518>
13. **RFC 7519**, “JSON Web Token (JWT),” Internet Engineering Task Force (IETF), 2015.
<https://datatracker.ietf.org/doc/html/rfc7519>
14. **C. Dwork and A. Roth**, “The Algorithmic Foundations of Differential Privacy,” Foundations and Trends in Theoretical Computer Science, 2014.
<https://www.cis.upenn.edu/~aaro/privacybook.html>
15. **S. Mondal et al.**, “Behavioral Biometrics for Continuous Authentication,” ACM Computing Surveys, 2019.
<https://dl.acm.org/doi/10.1145/3355615>

Web-Based AI-Secured User Authentication System : SAHAYAA

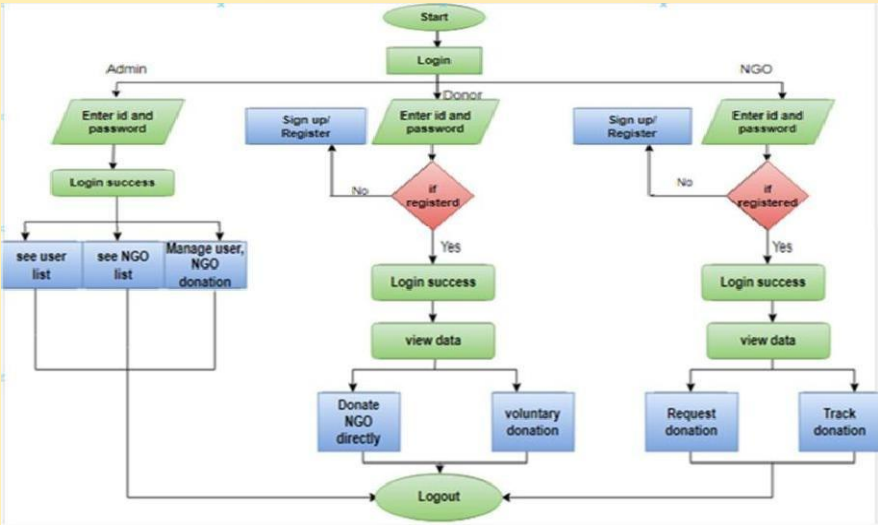


INTRODUCTION

Sahayaa is a modern web-based donation platform developed to simplify and digitalize the process of contributing to social causes in a secure and transparent manner. The platform connects donors with verified individuals, NGOs, and social campaigns, enabling users to make meaningful contributions with ease. This system integrates Artificial Intelligence techniques, to enhance authentication security by detecting abnormal login behavior, preventing unauthorized access, and strengthening identity verification processes. The SAHAYAA system aims to provide a more reliable, intelligent, and user-friendly authentication framework that improves security while maintaining ease of access for legitimate users.

AUTHORS
Mrs. Priyanka Chandragiri , K.Deepika , K.Neelaveni
K.Poojitha

ARCHITECTURE



RESULTS

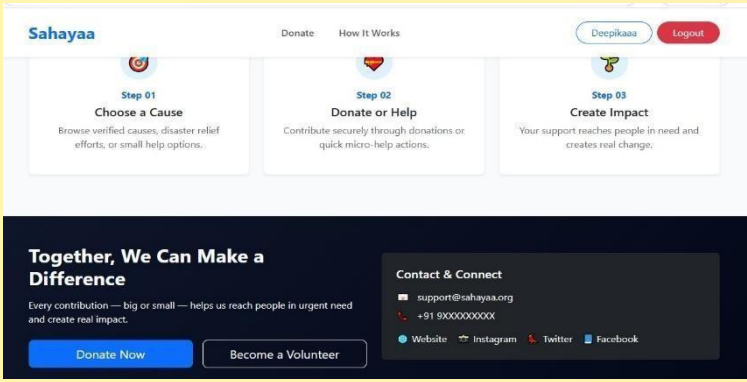


FIG:1 SHOWS THE PROJECT’S PURPOSE,TARGET BENEFICIARIES

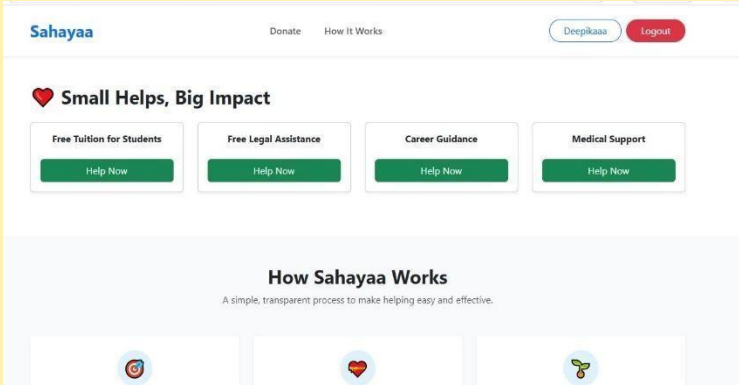


FIG:2 SHOWS THE PROJECT’S DONATION IMPACT WORKFLOW

OBJECTIVES

- Design an AI-secured web authentication framework
- Analyze user behavior patterns using ML
- Detect abnormal login activities in real time
- Reduce dependencies on static passwords
- Improve usability and security balances
- To analyze the monthly donation trends
- To identify peak and low donation periods for better planning of campaigns.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all the people who contributed in the completion of this project. I thank our guide to Continuously assess us and help us in this project we are also thankful to our research team members for the dedication collaboration and their hard work in conthbributing their time and effort in this project.

CONCLUSION:

REFERENCES

[1]A. Jain, A. Ross, and S. Prabhakar, "An Introduction to Biometric Recognition," *IEEE Transactions on Circuits and Systems for Video Technology* vol. 14, no. 1,pp. 4-20,2004.

[2] T. Mitchell, "Machine Learning and Cybersecurity Applications," *IEEE Security and Privacy Magazine*, vol. 16, no. 3, pp. 88–92, 2018.

[3] R. Sommer and V. Paxson, "Outside the Closed World: R. Sommer and V. Paxson, "Outside the Closed World: Detection," *IEEE Symposium on Security and Privacy*, pp. 305–316, 2010.

[4] National Institute of Standards and Technology (NIST), "Digital Identity Guidelines: Authentication and Lifecycle Management," Special Publication 800-63B, 2022.

[5] Shokri, R., et al., "Membership Inference Attacks Against Machine Learning Models," *IEEE on Security and Privacy*, 2017.

COMPONENTS:

HARDWARE COMPONENTS:

- Processor: Intel i3
- Ram: minimum 8 GB

Software Components:

- Operating System: Linux
- Development Tools: IDE
- Database: PostgreSQL
- Webserver: Apache
- Browser: Google chrome
- Security Tools: SSL/TLS

In conclusion SAHAYAA, an intelligent web-based user authentication system that integrates artificial intelligence with behavioral analysis to strengthen digital security. Unlike traditional authentication methods that rely solely on static credentials, SAHAYAA evaluates both user identity and interaction behavior to determine the legitimacy of each login attempt. By combining encrypted credential verification, real-time behavioral monitoring, and machine learning-based risk assessment, the system provides multi-layered protection against unauthorized access. against unauthorized access.

<https://github.com/PoojithaKonchada/web-based-ai-secure-user-authentication-system->

